POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

WILLIAM ZHANG
Département de Génie Électrique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Génie Électrique

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Sébastien LE DIGABEL, président
Antoine LESAGE-LANDRY, membre et directeur de recherche
Antoine LEGRAIN, membre

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

I gratefully acknowledge the Natural Sciences and Engineering Research Council of Canada (NSERC) for funding this research, and [source] for the scholarship that supported my stay at Stanford.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION . . . iii

ACKNOWLEDGEMENTS . . . iv

RÉSUMÉ . . . v

ABSTRACT . . . vi

LIST OF TABLES . . . ix

LIST OF FIGURES . . . x

LIST OF SYMBOLS AND ACRONYMS . . . xiii

CHAPTER 1 INTRODUCTION . . . 1
- 1.1 Problem statement . . . 2
- 1.2 Canonicalization . . . 4
  - 1.2.1 Conic reformulation . . . 6
  - 1.2.2 `CVXPY` . . . 7
  - 1.2.3 Example . . . 8
- 1.3 Computing derivatives . . . 9
  - 1.3.1 Numerical differentiation . . . 11
  - 1.3.2 Symbolic differentiation . . . 11
  - 1.3.3 Algorithmic differentiation . . . 12
- 1.4 Matrix computations . . . 13
  - 1.4.1 Dense matrices . . . 13
  - 1.4.2 Sparse matrices . . . 14
  - 1.4.3 Sparse matrix products . . . 15
- 1.5 Contributions . . . 16

CHAPTER 2 LITERATURE REVIEW . . . 18
- 2.1 Matrix stuffing in `CVXPY` . . . 18
  - 2.1.1 Composing linear operators . . . 19
  - 2.1.2 Example . . . 20
  - 2.1.3 Parameter caching . . . 21
- 2.2 Sparse derivatives from detected structure . . . 24
  - 2.2.1 Matrix-vector products and accumulation . . . 24

2.2.2 Compression by graph colouring . . . 26
2.2.3 `CasADi` . . . 28
2.3 Coefficient extraction from declared structure . . . 29
2.3.1 `AMPL` . . . 29
2.3.2 `JuMP` . . . 31

CHAPTER 3 A LIGHTWEIGHT CANONICALIZATION ENGINE . . . 33
3.1 Overview of the engine . . . 33
3.1.1 Atom classes . . . 34
3.1.2 The expression node . . . 34
3.2 Matrix representations . . . 35
3.2.1 The permuted dense matrix . . . 36
3.2.2 The stacked permuted dense matrix . . . 39
3.3 Computational kernels . . . 41
3.3.1 Kernel dispatch . . . 41
3.3.2 Reusing work across passes . . . 42
3.3.3 Problem assembly and parameter updates . . . 42
3.4 From `CVXPY` to the engine . . . 43

CHAPTER 4 BENCHMARK RESULTS . . . 47
4.1 Benchmark Setup . . . 47
4.1.1 Comparing like with like . . . 47
4.1.2 Two baselines for `cvxcore` . . . 48
4.1.3 Protocol and environment . . . 48
4.2 Non-parametric problems . . . 49
4.3 Parametric problems . . . 51
4.3.1 The cost of building the parameter tensor . . . 51
4.3.2 Cold construction, compared like with like . . . 52
4.3.3 When the parameter tensor repays itself . . . 53
4.4 Summary . . . 54

CHAPTER 5 CONCLUSION . . . 56
5.1 Summary of Works . . . 56
5.2 Limitations . . . 56
5.3 Future Research . . . 56

REFERENCES . . . 57

LIST OF TABLES

Table 1.1	A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output.	4
Table 2.1	A subset of the linear operators, classified as views or copies	20
Table 3.1	The atom classes and how each computes its Jacobian.	34
Table 4.1	Cold compilation time on the non-parametric problems	50
Table 4.2	Cost of building the parameter tensor, by backend	51
Table 4.3	Cold construction on the parametric problems	52
Table 4.4	Amortization of the parameter tensor	54

LIST OF FIGURES

Figure 1.1 Example of an affine expression tree for $\mathbf{Ax} - \mathbf{b}$. Variables are drawn as circles and constants (or parameters) as boxes. 5

Figure 1.2 Before canonicalization. `CVXPY` represents the DCP problem (1.5) as these expression trees. Variables are labelled with a circle, and parameters with a square. 8

Figure 1.3 After canonicalization. `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $\mathbf{x}$ appears as a leaf of several trees; all occurrences refer to the same variable. 10

Figure 1.4 The least squares objective $f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{2\times 2}$ as a scalar and as a vectorized computational graph. In the scalar graph every edge carries a scalar partial derivative. In the vectorized graph the local derivative of each node is a Jacobian block, shown below the node: $\mathbf{A}$ for the affine node and $2\mathbf{r}^\top$ for the sum of squares, where $\mathbf{r} = \mathbf{Ax} - \mathbf{b}$. 13

Figure 1.5 Dense storage of a 2×3 matrix: the same entries as one contiguous array in row-major and in column-major order. 14

Figure 1.6 The product $\mathbf{y} = \mathbf{Sv}$ with the matrix of (1.10). In CSR, the nonzeros of one row form a dot product that produces one output entry. In CSC, the nonzeros of one column are scaled by one input entry and accumulated into several output entries. 16

Figure 2.1 Jacobians under the vectorization (2.1), for a matrix variable $\mathbf{X}$ and the product $\mathbf{FX}$, with $\mathbf{X}, \mathbf{F} \in \mathbb{R}^{2\times 2}$. The entries of $\mathbf{vec}(\mathbf{X})$, in column-major order, index the columns of the Jacobians. 19

Figure 2.2 The column offsets of a stack of three variables $\mathbf{z} = (\mathbf{u}, \mathbf{v}, \mathbf{w})$ with $\mathbf{u} \in \mathbb{R}^2$, $\mathbf{v} \in \mathbb{R}^3$, and $\mathbf{w} \in \mathbb{R}^2$. Each offset is the total dimension of the preceding variables: $\mathbf{u}$ starts at column 0, $\mathbf{v}$ at 2, and $\mathbf{w}$ at 5. The entries of $\mathbf{z}$ index the columns of every augmented matrix $[\mathbf{A}_f \ \mathbf{b}_f]$. . . 21

Figure 2.3 Matrix stuffing of problem (1.5) by a bottom-up pass over the conic-form expression trees of Figure 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children. The coefficients appear negated because (1.2) writes the constraints as $\mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}$. The horizontal rules in $[\mathbf{A}\ \mathbf{b}]$ separate the rows of the three constraints. 22

Figure 2.4 What each layout stores during the pass, at the node $-(\mathbf{F}\mathbf{x} - \mathbf{g})$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns. 22

Figure 2.5 The DPP path on the objective data of problem (1.5): the parameter map $\mathbf{Q}$ is built and cached once, and each re-solve recovers $\mathbf{c}$ with the single sparse product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$. 23

Figure 2.6 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the pass is repeated on the parameter-free tree on every change of the parameters. 23

Figure 2.7 Forward sweep of a computational graph 25

Figure 2.8 Reverse sweep of a computational graph 25

Figure 2.9 Accumulation by vertex elimination 26

Figure 2.10 Graph colouring compresses the seeds 27

Figure 3.1 The structure of the Jacobian of $\mathbf{B}\mathbf{x}$ with respect to the full variable vector, for dense data $\mathbf{B} \in \mathbb{R}^{50\times 50}$ in a problem with ten thousand variables. The nonzeros are exactly the entries of $\mathbf{B}$, and the width of the matrix is set by the problem rather than by the expression. . . . 36

Figure 3.2 A permuted dense matrix. The dense block $\mathbf{X}$ is stored once, contiguously; the index vectors $\mathbf{r}$ and $\mathbf{c}$ place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored. Each block row keeps its own shade, reused in Figure 3.4. 37

Figure 3.3 The two selector matrices of the example of Figure 3.2, with $\mathbf{r} = (1, 3, 4)$ and $\mathbf{c} = (0, 2, 5)$. The rows of $\mathbf{S}_r$ are the rows of the 6×6 identity indexed by $\mathbf{r}$, and the rows of $\mathbf{S}_c$ are the rows of the 7×7 identity indexed by $\mathbf{c}$. Their product with $\mathbf{X}$ is the $\mathbf{A}$ of Figure 3.2; column i of $\mathbf{S}_r^\top$ carries block row i to global row r_i, so the shades match. Neither selector is ever formed. 37

Figure 3.4 Row indexing of the permuted dense matrix $\mathbf{A}$ of Figure 3.2. The table traces the scan of the inverse row map: each requested global row yields a block row of $\mathbf{X}$ to copy, or -1, in which case the output row is structurally zero and not stored. Shades follow the block rows of Figure 3.2. The result is the permuted dense triple $(\mathbf{X}_C, \mathbf{r}_C, \mathbf{c}_C)$. . 38

Figure 3.5 The Jacobian of $\mathbf{AX}$ as a stacked permuted dense matrix, for dense $\mathbf{A} \in \mathbb{R}^{2\times 2}$ and a variable $\mathbf{X} \in \mathbb{R}^{2\times 2}$ in a problem with six variables. There is one block per column of $\mathbf{X}$, each a permuted dense copy of $\mathbf{A}$ at its own rows and columns. The 4×6 matrix $\mathbf{M}$ is never assembled; a product with $\mathbf{M}$ is one `dgemm` per block. 40

Figure 3.6 Double dispatch of the transposed products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{DA}$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its colour indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel. 41

Figure 3.7 The two passes of the `sum` atom. During the symbolic pass, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The numeric passes of the other structural atoms have the same form. 43

Figure 3.8 The conic form of the running example (1.5), whose expression trees are in Figure 1.3, after conversion. The atoms are the engine nodes, the circles are variables and the boxes are parameters; `left_matmul` holds the data matrix $\mathbf{F}$ as a dense block. The variable $\mathbf{x}$ is converted once and reached from three places, so the four trees form one graph. The table lists the roots in the order the constraints are grouped, which is the order of the row blocks of $[\mathbf{A}\ \mathbf{b}]$ in Figure 2.3. 45

LIST OF SYMBOLS AND ACRONYMS

AD	Algorithmic Differentiation
AML	Algebraic Modelling Language
AMPL	A Mathematical Programming Language
ASA	Affine-Solver-Affine
ASL	AMPL Solver Library
BLAS	Basic Linear Algebra Subprograms
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
JVP	Jacobian-Vector Product
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SIMD	Single Instruction, Multiple Data
SOCP	Second-Order Cone Program
SDP	Semidefinite Program
VJP	Vector-Jacobian Product

CHAPTER 1 INTRODUCTION

Since the early days of linear programming, optimization practitioners have needed to translate mathematical formulations into data that solvers can process. At first, this meant encoding each coefficient of the problem matrices by hand [1]. Matrix generators were introduced in the late 1960s to automate this task: they compute the coefficients through explicit loops and write them out in solver-readable formats like MPS [2, 3]. Matrix generators were able to remove most of the manual labour, but a gap still remained. Modellers naturally think in terms of sets, indices, and algebraic expressions, while procedural code constructs the coefficient matrices entry by entry. This mismatch between how a model is expressed and how it is implemented made models difficult to verify, modify, and communicate with others [4]. Algebraic modelling languages such as `AMPL` [5], `AIMMS` [6], and `GAMS` [7] closed this gap by accepting problem descriptions that closely follow the mathematical formulation. The survey [8] provides a complete overview of the early history of modelling languages.

More recently, modelling languages embedded in general-purpose programming languages have gained popularity. These include `Pyomo` [9,10] in Python, `JuMP` [11] in Julia, `CasADi` [12] in C++, and `YALMIP` [13] in MATLAB. Other domain-specific modelling languages specialized for convex optimization [14] are built on disciplined convex programming (DCP), the composition rules under which the convexity of a problem can be verified automatically [15, 16]. These DCP-based languages include `CVX` [17] in MATLAB, `CVXR` [18] in R, `CVXPY` [19] in Python, and `Convex.jl` [20] in Julia.

In the meantime, modern general-purpose solvers [21–23] have mostly transitioned towards conic programming, which generalizes linear programming and can represent and solve nearly every convex program used in practice [24]. The translation task remains a responsibility of the modelling languages, which now target conic problem data instead of Linear Program (LP) coefficient matrices. In DCP-based languages, this translation process is called canonicalization [25]. While the time taken to solve an optimization problem is often dominated by the solver, the time taken to canonicalize (or compile) it can also be significant [26]. The main goal of this Master's thesis is to reduce the compilation time of `CVXPY`.

1.1 Problem statement

The standard form of a convex optimization problem is

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & f_0(\mathbf{x};\boldsymbol{\theta}) \\ \text{subject to} & f_i(\mathbf{x};\boldsymbol{\theta}) \le 0, \quad i = 1,\dots,m_1 \\ & g_j(\mathbf{x};\boldsymbol{\theta}) = 0, \quad j = 1,\dots,m_2 \end{array} \tag{1.1}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the optimization variable, the objective $f_0 : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ and inequality constraint functions $f_1, \dots, f_{m_1} : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ are convex, and the equality constraint functions $g_1, \dots, g_{m_2} : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}$ are affine [14]. The parameter $\boldsymbol{\theta} \in \mathbb{R}^p$ specifies data that can change between solves but is fixed and given when solving an instance of the problem. We call the parametrized problem (1.1) a problem family; for a fixed value of $\boldsymbol{\theta}$, we call it a problem instance.

Canonicalization converts the high-level description (1.1) to a quadratic conic form used by many general-purpose convex solvers

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \frac{1}{2}\mathbf{x}^\top \mathbf{P}(\boldsymbol{\theta})\mathbf{x} + \mathbf{c}(\boldsymbol{\theta})^\top \mathbf{x} \\ \text{subject to} & \mathbf{b}(\boldsymbol{\theta}) - \mathbf{A}(\boldsymbol{\theta})\mathbf{x} \in \mathcal{K} \end{array} \tag{1.2}$$

where the *problem data* are the symmetric positive semidefinite matrix $\mathbf{P}(\boldsymbol{\theta}) \in \mathbb{S}^n_+$, the matrix $\mathbf{A}(\boldsymbol{\theta}) \in \mathbb{R}^{m\times n}$, the vectors $\mathbf{b}(\boldsymbol{\theta}) \in \mathbb{R}^m$ and $\mathbf{c}(\boldsymbol{\theta}) \in \mathbb{R}^n$, and the non-empty closed convex cone $\mathcal{K} \subseteq \mathbb{R}^m$. Problem (1.2) generalizes both quadratic programs (QPs), when $\mathcal{K} = \mathbb{R}^m_+$ is the nonnegative orthant, and standard conic programs, when $\mathbf{P}(\boldsymbol{\theta}) = 0$.

Most of the cost of canonicalization lies in the final step, where symbolic expression trees are converted to numerical problem data. This step is often called matrix stuffing or the backend of canonicalization [27]. During the first solve of a problem family, `CVXPY` forms and caches a parameter tensor that maps the parameter vector to the problem data. On subsequent solves, new problem data can be obtained with a single matrix-vector product [26].

For large-scale problems, forming the parameter tensor can be prohibitively expensive, so `CVXPY` lets users opt out of it entirely. Users must thus choose between paying a large upfront cost on the first solve and repeating the full canonicalization, with nothing reused, at every solve. We review `CVXPY` and the frontend of canonicalization in §1.2, as well as the parameter tensor and matrix stuffing in §2.1.

The constraint matrix $\mathbf{A}$ is the Jacobian of the stacked constraint expressions and $\mathbf{P}$ is the Hessian of the objective, so matrix stuffing amounts to computing a sparse Jacobian and

Hessian. Classical methods build these matrices column by column with multiple passes of algorithmic differentiation (AD), using graph colouring to group columns that can be recovered in a single pass [28, 29]. These methods treat the expression as a blackbox: the sparsity pattern must first be detected and the columns grouped before a single entry of the matrix is obtained. We review the basics of computing derivatives in §1.3 and present sparse AD and graph colouring methods in §2.2.

A modelling language, by contrast, knows which variables enter each constraint. `AMPL` (A Mathematical Programming Language) is split into two components: its commercial translator writes the structure of the model to a `.nl` file [30], and the open source AMPL Solver Library (ASL) evaluates constraints and their derivatives from this file inside the solver process [31]. `Pyomo` also interfaces to solvers through `.nl` files, among other formats, though it generally writes them more slowly than the `AMPL` translator.

On the other hand, `JuMP` has no single standard form to target; it decouples modeller and solver through MathOptInterface (MOI), a typed in-memory interface [32]. The model is built constraint by constraint, and each constraint stores its coefficients directly, so extraction reduces to copying them into the format of the chosen solver. None of these approaches operates on expression trees: the AD tools record the expression as a flat tape of instructions, `AMPL` exchanges the model through a file, and `JuMP` stores lists of coefficients. In `CVXPY` the expression trees themselves are the input of the backend. We review both these modelling languages in §2.3.

Our approach. We make use of the fact that the sparsity patterns of $\mathbf{A}$ and $\mathbf{P}$ are fixed across a problem family, because the parameter $\boldsymbol{\theta}$ changes the values of the problem data but not their structure. We can therefore split matrix stuffing into a symbolic and a numeric phase. The symbolic phase runs once on the first solve, where it computes the sparsity patterns, allocates memory for all intermediate matrices, and caches other computations that remain valid across the problem family. The numeric phase then fills in new values at every solve. This separation between structure and values is inspired by the interfaces of Nonlinear Program (NLP) solvers such as `IPOPT` [33, 34].

The symbolic phase is much cheaper to form and cache than the parameter tensor, and the numeric phase is often similar in cost to applying the matrix-vector product. The problem data is also assembled entirely in memory rather than through a separate model file. Finally, we exploit structure within the sparsity patterns themselves, which often contain dense blocks of data. When possible, we store these blocks as dense matrices and compute with dense linear algebra kernels, rather than always relying on generic sparse formats and routines (we

provide background on dense and sparse matrix computations in §1.4).

1.2 Canonicalization

The problem family (1.1) is written in functional form, as objective and constraint functions of the variable and the parameters. In a modelling language, the optimization variables $\mathbf{x}_1, \ldots, \mathbf{x}_k$ are collected into a single vector $\mathbf{x} \in \mathbb{R}^n$, and likewise the parameters $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_l$ into $\boldsymbol{\theta} \in \mathbb{R}^p$ [14]. Users are not required to write constraints with a zero right-hand side as in (1.1). Modelling languages accept inequalities with a convex left-hand side and a concave right-hand side, and equalities with both sides affine. Converting to the form (1.1) is trivial and only requires moving every term to the left-hand side. Internally, each function is represented as an *expression tree*.

Modelling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in `CVXPY` because they are the fundamental building blocks of the modelling language [15]. Alongside its numerical definition, each atom declares useful attributes such as its curvature, monotonicity with respect to each argument, and the sign of its output. A selection of common atoms is shown in Table 1.1.

Table 1.1 A selection of `CVXPY` atoms with their DCP attributes: curvature, monotonicity in each argument, and sign of the output.

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`exp`	e^x	convex	non-decreasing	positive
`log`	$\log x$	concave	non-decreasing	any
`norm`	$\lVert x \rVert_2$	convex	unknown	nonnegative
`max`	$\max_i x_i$	convex	non-decreasing	any
`sum_largest`	sum of k largest entries	convex	non-decreasing	any

An expression is defined recursively as an atom evaluated at subexpressions, each of which is itself another expression. Such an expression is naturally stored as an expression tree, with atoms at its internal nodes and variables, parameters, and constants at its leaves. In Python, this tree can be formed automatically from user code through operator overloading. Most operations on an expression, from evaluation to verifying convexity to differentiation,

traverse the tree and combine the intermediate results at each node.

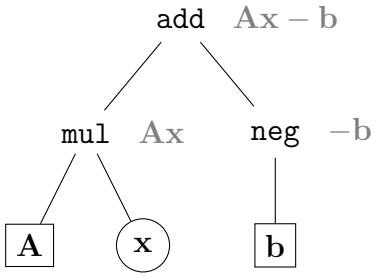


Figure 1.1 Example of an affine expression tree for $\mathbf{Ax} - \mathbf{b}$. Variables are drawn as circles and constants (or parameters) as boxes.

To certify the convexity of a problem, DCP-based modelling languages use the DCP composition rules to classify the curvature of each function [15]. Once all functions are classified, a problem is DCP-compliant if the objective and each constraint have curvatures compatible with a convex problem: a convex objective, convex $\leq$ concave inequalities, and affine equalities. Otherwise, if a curvature is incompatible (e.g., f_0 is concave) or cannot be determined, the modelling language raises an error and terminates. Because these rules are only sufficient for convexity, a problem that is genuinely convex but not expressed in a recognized form may still be rejected.

The composition $\phi(u_1(\mathbf{x}), u_2(\mathbf{x}), \ldots, u_k(\mathbf{x}))$ of an atom $\phi : \mathbb{R}^k \to \mathbb{R}$ with subexpressions $u_i : \mathbb{R}^n \to \mathbb{R}$ is convex when ϕ is convex and, for each argument $i \in \{1, \ldots, k\}$, one of the following holds:

- ϕ is non-decreasing in argument i and u_i is convex;
- ϕ is non-increasing in argument i and u_i is concave; or
- u_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require ϕ and all of its arguments to be affine [15].

1.2.1 Conic reformulation

Once a problem is verified to be DCP-compliant, canonicalization rewrites its functional form (1.1) into an equivalent functional conic form

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & f_0(\mathbf{x};\boldsymbol{\theta}) \\ \text{subject to} & h_i(\mathbf{x};\boldsymbol{\theta}) \in \mathcal{K}_i, \quad i = 1,\dots,m, \end{array} \tag{1.3}$$

where f_0 is affine or quadratic, each $h_i : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^{d_i}$ is affine, and each $\mathcal{K}_i \subseteq \mathbb{R}^{d_i}$ is one of the standard cones supported by general-purpose conic solvers: the zero, nonnegative, second-order, power, exponential, and positive semidefinite cones.

Every nonlinear atom is rewritten into conic constraints in three steps. The first step rewrites the problem into *Smith form*, where each nonlinear atom is replaced by a new variable coupled with an equality constraint; every atom then acts only on affine arguments [35]. For a convex atom ϕ, the Smith form gives an auxiliary variable $t \in \mathbb{R}$ with the equality constraint $\phi(\mathbf{x}) = t$. The second step relaxes this equality to the inequality $\phi(\mathbf{x}) \le t$. The DCP rules allow a convex atom only where its value is minimized or upper-bounded, so the solver always prefers t as small as possible; the relaxation is therefore lossless. The pair $(\mathbf{x}, t)$ then lies in the *epigraph* $\{(\mathbf{x}, t) \in \mathbb{R}^n \times \mathbb{R} : \phi(\mathbf{x}) \le t\}$ of ϕ, so we often refer to t as an epigraph variable. A concave atom relaxes to $\phi(\mathbf{x}) \ge t$ instead, giving a hypograph variable. The third and final step expresses each epigraph (hypograph) inequality as the membership of an affine function of $(\mathbf{x}, t)$ in a convex cone [36].

Example of reformulations. A first example is the ℓ_2 norm atom $\|\mathbf{x}\|_2$ of $\mathbf{x} \in \mathbb{R}^n$, whose epigraph $\|\mathbf{x}\|_2 \le t$ is exactly the second-order cone constraint $(t, \mathbf{x}) \in \mathcal{K}_{\text{soc}}$, where $\mathcal{K}_{\text{soc}} = \{(t, \mathbf{x}) \in \mathbb{R} \times \mathbb{R}^n : \|\mathbf{x}\|_2 \le t\}$. A second example is the nondifferentiable absolute value atom $|x|$ of a scalar x, whose epigraph $|x| \le t$ can be written as the pair of affine inequalities $-t \le x \le t$. A final example is the sum of the k largest entries of $\mathbf{x}$, which expands into a small LP: the epigraph sum_largest$(\mathbf{x}, k) \le t$ holds if and only if there exist a scalar $q \in \mathbb{R}$ and a vector $\mathbf{w} \in \mathbb{R}^n$ such that

$$q\mathbf{1} + \mathbf{w} \ge \mathbf{x}, \qquad \mathbf{w} \ge 0, \qquad kq + \mathbf{1}^\top \mathbf{w} \le t, \tag{1.4}$$

where the inequalities are applied element-wise. Here q acts as a threshold and each w_i measures how far the entry x_i rises above it, so that the sum of the k largest entries is bounded by $kq + \mathbf{1}^\top \mathbf{w}$ [14].

1.2.2 CVXPY

CVXPY is a DCP-based modelling language embedded in Python [19], which began in 2013 as an implementation of the modelling principles of CVX [17]. It differed from CVX in three design decisions: modular components built around a few shared data structures, the expression tree formed by Python's operator overloading as the central representation, and support for parameters, which lets compilation work be reused when a problem is solved repeatedly. These ideas matured with version 1.0, which redesigned canonicalization as a general reduction-based rewriting system [25]. The new design enabled the addition of other disciplined grammars [34,37,38], and the early parameter support grew into DPP, a restricted grammar under which canonicalized programs can be updated efficiently [26].

The open source project has since grown through contributions from many other enthusiastic developers; a recent paper surveys the new canonicalization and modelling features introduced between versions 1.1 and 1.9 [27].

The rewriting from the functional form (1.1) to the functional conic form (1.3) is carried out as a sequence of *reductions*. Each reduction provides an `apply` step that converts a problem into an equivalent one, and a `retrieve` step that recovers a solution of the original problem from a solution of the rewritten one [25]. A first example is `FlipObjective`, which turns a maximization problem into a minimization one by flipping the sign of the objective, and flips it back in its `retrieve` step. A second example is `CvxAttr2Constr`, which replaces variable attributes such as nonnegativity with explicit constraints when the target solver does not support them natively.

The main reduction, `Dcp2Cone`, applies the epigraph reformulation of §1.2.1 in a bottom-up pass over the expression trees. The problem is first rewritten into Smith form, each resulting equality is relaxed to an epigraph (or hypograph) inequality, and each inequality is finally expressed as a cone membership [35, 36]. In practice the three steps are performed at once: each nonlinear atom is replaced directly by its epigraph variable and cone constraints, and the intermediate Smith form and its relaxation never appear explicitly.

When the problem is a quadratic program and a solver that accepts a quadratic objective is available, the reduction `Qp2SymbolicQp` is applied in place of `Dcp2Cone`. It rewrites each quadratic atom (a squared power, a quadratic over a linear term, a Huber penalty) as a quadratic form in a new variable, and adds the equality constraint that sets this variable equal to the affine argument of the atom. The objective is then a sum of quadratic forms with constant matrices, and the constraints remain affine [63].

1.2.3 Example

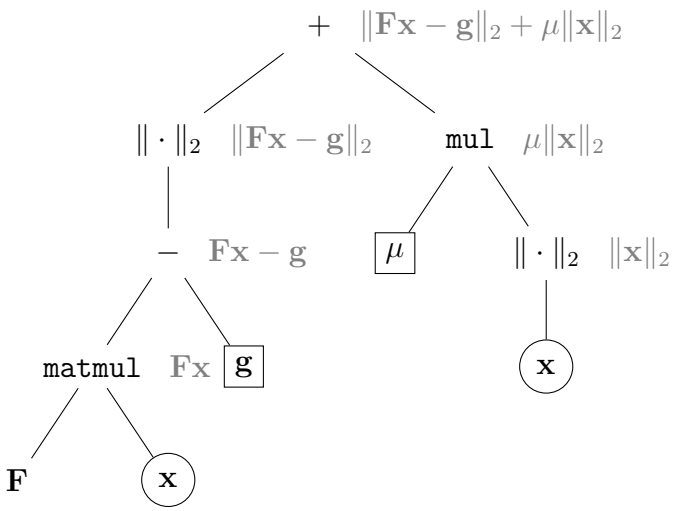
We consider the following simple fitting with regularization problem

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \|\mathbf{F}\mathbf{x}-\mathbf{g}\|_2 + \mu\|\mathbf{x}\|_2 \\ \text{subject to} & \mathbf{x} \geq 0 \end{array} \tag{1.5}$$

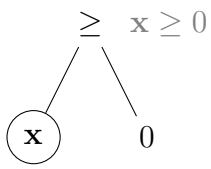
with variable $\mathbf{x} \in \mathbb{R}^n$, fixed problem data $\mathbf{F} \in \mathbb{R}^{m\times n}$, and parameters $\mathbf{g} \in \mathbb{R}^m$ and $\mu \geq 0$; the constraint $\mathbf{x} \geq 0$ requires every entry of $\mathbf{x}$ to be nonnegative. The problem formulation in `CVXPY` code is

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n)
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

where we also use the numerical computation library `NumPy` [39] for the data arrays. The expression tree representation of the problem is shown in Figure 1.2.



(a) Expression tree of the objective



(b) Expression tree of the constraints

Figure 1.2 Before canonicalization. `CVXPY` represents the DCP problem (1.5) as these expression trees. Variables are labelled with a circle, and parameters with a square.

After applying the reductions, the problem has been canonicalized to

$$\begin{array}{ll} \underset{t_1,\, t_2,\, \mathbf{x}}{\text{minimize}} & t_1 + \mu t_2 \\ \text{subject to} & \mathbf{x} \geq 0 \\ & (t_1, \mathbf{F}\mathbf{x} - \mathbf{g}) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, \mathbf{x}) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{1.6}$$

where $t_1, t_2 \in \mathbb{R}$ are new epigraph variables, and the cone $\mathcal{K}$ from (1.2) is the product $\mathbb{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$. Problem (1.6) is now in the functional conic form of §1.2.1: every constraint is a cone membership, and the objective and constraint expressions are affine. Comparing the DCP trees of Figure 1.2 with the trees of Figure 1.3 shows the effect of canonicalization: the nonlinear norm atoms have been replaced by epigraph variables, leaving affine expression trees in the variables, which we call the conic-form trees of the problem. The three rows of (1.6) remain three separate cone constraints, each carrying the affine expressions that must lie in its cone; Figure 1.3 accordingly shows one expression tree per constraint, rather than a single combined tree.

1.3 Computing derivatives

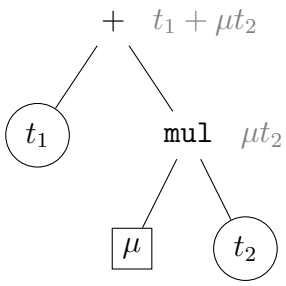
Computing derivatives is a fundamental task in scientific computing, with many methods that differ in cost, in accuracy, and in the form of the result [40]. The derivatives in this Master's thesis are the constraint Jacobian $\mathbf{A}$ and the objective Hessian $\mathbf{P}$ of a canonicalized problem, both large sparse matrices. This section presents the Jacobian and the Hessian, the elementary methods for computing them, and the choices that distinguish the software implementations; §2.2 reviews sparse AD and graph colouring methods.

For a vector-valued map $F : \mathbb{R}^n \to \mathbb{R}^m$ of an input $\mathbf{x} \in \mathbb{R}^n$, the first-order derivative is the Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ of partial derivatives of the outputs with respect to the inputs,

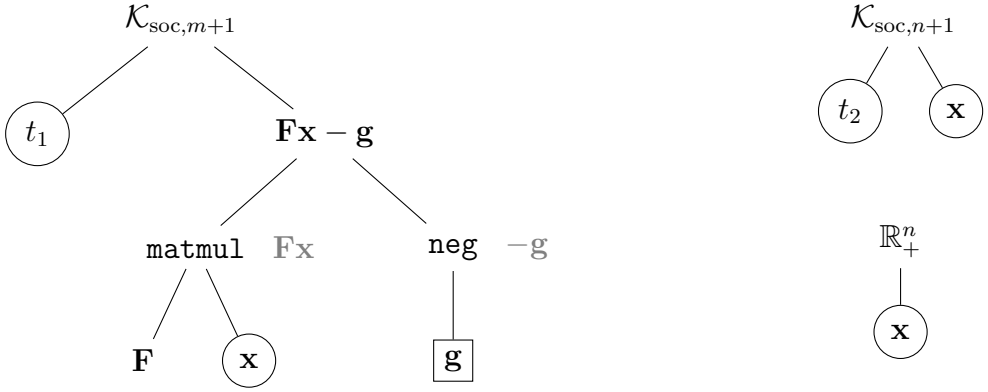
$$J_{ij} = \frac{\partial F_i}{\partial x_j}, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n. \tag{1.7}$$

We mainly consider the constraint function, which stacks the potentially many affine expressions of a canonicalized problem into a single vector function of the variables. Its Jacobian is the constant constraint matrix $\mathbf{A}$ of (1.2), whose sparsity pattern is fixed although its values may change from one solve to the next.

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the second-order derivative is the Hessian $\mathbf{H} \in \mathbb{R}^{n \times n}$ of



(a) Expression tree of the objective



(b) One expression tree per constraint

Figure 1.3 After canonicalization. `CVXPY` represents the conic form (1.6) as these expression trees: the nonlinear norm atoms have become epigraph variables, leaving the objective and the three separate cone constraints affine. The variable $\mathbf{x}$ appears as a leaf of several trees; all occurrences refer to the same variable.

second partial derivatives,

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}, \qquad i, j = 1, \ldots, n. \tag{1.8}$$

The Hessian is symmetric whenever f is twice continuously differentiable, and it is then the Jacobian of the gradient of f. It can therefore be obtained by differentiating the gradient with the same methods used for the Jacobian, although dedicated methods that compute it directly also exist. The function we consider is the canonicalized objective, whose Hessian is the matrix $\mathbf{P}$ of (1.2).

1.3.1 Numerical differentiation

Other than computing derivatives analytically, the simplest method is to use numerical differentiation, which approximates the derivative of a function at a point by evaluating the function at nearby points using a finite difference formula such as

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{1.9}$$

where h is a small step size. Note that the central variant $f'(x) \approx (f(x+h) - f(x-h))/2h$ improves accuracy. Both formulas use two function evaluations, but the forward difference can reuse the already computed value $f(x)$ across directions, while the central variant cannot. The approach is simple but has two downsides. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to truncation errors, when h is too large, or rounding errors, when h is too small, in the computed derivative if the step size is not carefully chosen [41]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations, also known as a derivative checker [42].

1.3.2 Symbolic differentiation

Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This symbolic approach to differentiation is used in computer algebra systems such as `Mathematica` [43] and `Maple` [44] as well as the `SymPy` library in Python [45]. Unlike its numerical counterpart, symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving

analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can quickly form exponentially large symbolic expressions, which is impractical and inefficient for many problems [46]. When intermediate expressions are stored as common subexpressions, and the expression is represented with a directed acyclic graph (DAG) instead of a tree, symbolic differentiation performs the same set of operations as reverse mode differentiation [47].

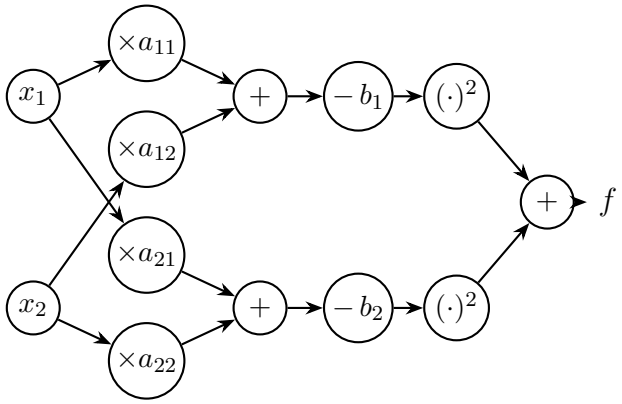
1.3.3 Algorithmic differentiation

Algorithmic differentiation (AD) treats the function as a computational graph of elementary operations, each with a known local derivative rule. The chain rule is then applied to combine these local derivatives throughout the graph, giving the derivative of the whole function. AD is also commonly referred to as automatic differentiation. We prefer the term algorithmic to emphasize that AD differentiates the algorithm that computes a function, rather than a numerical approximation or a symbolic formula [46].

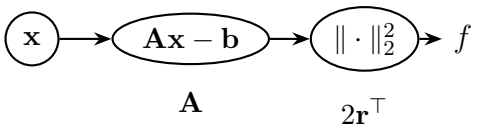
In this broad sense, AD can refer to any method that applies the chain rule over a computational graph. Its implementations differ in four basic choices: whether the graph is scalar or vectorized, whether the chain rule runs forward or in reverse, whether the derivative is assembled directly or accessed through matrix–vector products, and whether the result is stored sparse or dense.

In most AD tools, every node is a single elementary operation, an addition, a multiplication, or a univariate function, and every edge carries a scalar partial derivative [46]. In a vectorized graph, a node is a whole vector or matrix operation, and its local derivative is a matrix, a block of the Jacobian. Figure 1.4 contrasts the two on a small least squares objective. `CasADi` [12] offers both, through its scalar SX and vectorized MX expressions.

The modes of AD differ in the order in which the local derivatives are accumulated. Forward mode (or tangent mode) propagates derivatives from the inputs toward the outputs, following the order of evaluation. Reverse mode (or adjoint mode) propagates them from the outputs back toward the inputs, and requires storing the intermediate values of the evaluation [40]. In forward mode the work scales with the number of input variables, and in reverse mode with the number of output variables.



(a) Scalar graph: one node per elementary operation



(b) Vectorized graph: one node per matrix operation

Figure 1.4 The least squares objective $f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{2\times 2}$ as a scalar and as a vectorized computational graph. In the scalar graph every edge carries a scalar partial derivative. In the vectorized graph the local derivative of each node is a Jacobian block, shown below the node: $\mathbf{A}$ for the affine node and $2\mathbf{r}^\top$ for the sum of squares, where $\mathbf{r} = \mathbf{Ax} - \mathbf{b}$.

1.4 Matrix computations

The derivatives of §1.3 are matrices, and every matrix is stored in either a dense representation, which keeps all entries in a single array, or a sparse representation, which keeps only the nonzero entries alongside their positions. The representation determines the memory a matrix occupies and the algorithms that operate on it, and each has its own use cases. This section reviews the storage formats, algorithms, and some of the history behind both types of matrices.

1.4.1 Dense matrices

A dense matrix is stored as a single contiguous array of its entries, ordered row by row (row-major) or column by column (column-major), as shown in Figure 1.5. The position of an entry is computed directly from its indices, so no index arrays are needed, and rows or columns are traversed sequentially in memory. C arrays are row-major and Fortran arrays are column-major. Most dense linear algebra interfaces follow the Fortran convention.

Dense linear algebra is standardized by the Basic Linear Algebra Subprograms (BLAS), an interface of vector and matrix kernels [48, 49]. The BLAS is a specification rather than an implementation. Implementations such as `OpenBLAS`, Intel `MKL`, and Apple `Accelerate` are specialized for their respective processors, and nearly every machine has one available, whether shipped with the operating system or bundled by numerical libraries. LAPACK, the

$$\mathbf{D} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

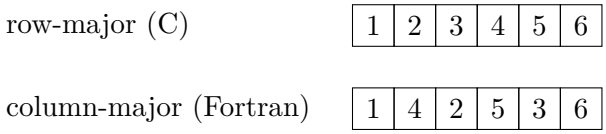


Figure 1.5 Dense storage of a 2×3 matrix: the same entries as one contiguous array in row-major and in column-major order.

library behind dense factorizations and linear system solving in most scientific software, is built on top of the BLAS kernels [50].

The BLAS interface is organized in three levels by the ratio of computation to data. Level 1 covers vector operations such as dot products, scaled copies, and additions, level 2 covers matrix–vector products, and level 3 covers matrix–matrix products. The routine names abbreviate the precision and the operation. For example, `dgemv` is the double-precision general matrix–vector product, and `dgemm` is the double-precision general matrix–matrix product.

BLAS implementations are fast because they are organized around the memory hierarchy. A modern processor multiplies numbers far faster than it loads them from main memory, so optimized kernels reuse data already in cache and process several entries per instruction with Single Instruction, Multiple Data (SIMD) units [51]. Level-3 kernels benefit the most because they perform a cubic number of operations on a quadratic amount of data, so each entry loaded from memory is reused many times.

1.4.2 Sparse matrices

In optimization problems, the matrices $\mathbf{P}$ and $\mathbf{A}$ often contain mostly zero entries. Such matrices are called sparse, and conic solvers expect their problem data in sparse formats. Techniques that store and compute with only the nonzero entries date back to the linear programming computations of the 1950s and the work of Markowitz [52]. The simplest sparse matrix data structure is the coordinate or triplet format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of nonzero entries in the matrix. For example, the matrix

$$\mathbf{S} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix} \tag{1.10}$$

can be represented in coordinate format as

```
int    rows[] = {0, 1, 1,  1,  2,  2};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we use zero-based indexing and array declaration notation from the C programming language [53]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices. The compressed sparse row (CSR) format is commonly used instead.

The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (1.10) can be represented in CSR format as

```
int  idxptr[] = {0, 1,         4,      6};
int    cols[] = {1, 0, 2,  3,  1,  2};
double vals[] = {1, 2, 3, 4,  5,  6};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very cache-friendly because the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for iterative methods to solve sparse linear systems [54].

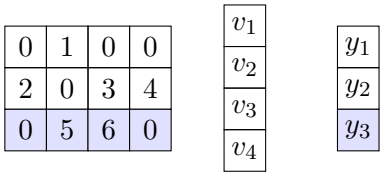
The compressed sparse column (CSC) format applies the same compression to the columns instead. The pointer array `idxptr` now points into a `rows` array, and the nonzero entries are stored column by column. The CSC arrays of a matrix read exactly as the CSR arrays of its transpose, so transposition is a conversion between the two formats, computable without sorting in one counting pass over the nonzeros [55]. Accessing a column of a CSR matrix instead requires a scan of every row, so algorithms that traverse columns prefer CSC; direct methods for sparse linear systems are the standard example [56].

The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [57] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

1.4.3 Sparse matrix products

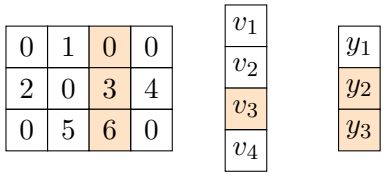
Both compressed formats multiply a dense vector in one pass over the nonzeros, but the loop must follow the storage. CSR gathers, computing each output entry as a sparse dot product

along its row, while CSC scatters, accumulating each column's contribution into the output vector. Figure 1.6 traces one product of the matrix in (1.10) under both formats.



$y_3 = 5v_2 + 6v_3$

(a) CSR gathers along a row



$y_2 \mathrel{+}= 3v_3, \quad y_3 \mathrel{+}= 6v_3$

(b) CSC scatters a column

Figure 1.6 The product $\mathbf{y} = \mathbf{S}\mathbf{v}$ with the matrix of (1.10). In CSR, the nonzeros of one row form a dot product that produces one output entry. In CSC, the nonzeros of one column are scaled by one input entry and accumulated into several output entries.

The product of two sparse matrices is formed row by row with Gustavson's algorithm [55]. Each row of the result is a combination of rows of the second operand, weighted by the nonzeros of the corresponding row of the first. Because the sparsity pattern of the result is not known in advance, the row is summed in a dense work array and then written back to sparse form [58]. It is the standard sparse matrix product, implemented in popular packages such as `Eigen` [59] and `scipy.sparse`.

In contrast to the BLAS kernels of §1.4.1, sparse products access memory irregularly. The loops traverse the index arrays, so every nonzero costs an index load and an indirect access. Contiguous reads, SIMD, and cache tiling do not apply. Sparse formats are also a poor fit for dense data. A dense block stored in coordinate format triples its storage, one index pair per entry, and multiplying it through the sparse loop is far slower than the equivalent BLAS call. A sparse BLAS standard was specified by the BLAS Technical Forum [60], but implementations are few and far less widely adopted than the dense BLAS.

1.5 Contributions

We make the following contributions:

1. We build `SparseDiffEngine`, a C library with Python bindings that assembles sparse Jacobians and Hessians directly from the expression structure, with no sparsity detection or graph colouring.

2. We implement a canonicalization backend for `CVXPY` built on `SparseDiffEngine`; this

backend avoids the large upfront cost of the parameter tensor while still caching computations for subsequent solves.

3. We benchmark `SparseDiffEngine` against `CVXPY`'s existing canonicalization backends.

All code contributions are open source and available at

`https://github.com/SparseDifferentiation/SparseDiffengine`
`https://github.com/SparseDifferentiation/Jacobian-accumulation`

The rest of this thesis is organized as follows. Chapter 2 reviews how `CVXPY` and other modelling languages compute the problem data. In Chapter 3 we present an overview of the main data structures and algorithms in `SparseDiffEngine` as well as its interface in `CVXPY`. Finally, Chapter 4 benchmarks this engine against `CVXPY`'s three canonicalization backends, and finally, we conclude with a summary of our contributions and future improvements in Chapter 5.

CHAPTER 2 LITERATURE REVIEW

This chapter reviews the implementation of `CVXPY`'s matrix stuffing (§2.1) and of other modelling language approaches, from sparse AD such as `CasADi` (§2.2) to coefficient extraction in `AMPL` and `JuMP` (§2.3). These modelling languages all produce the same output as `CVXPY`, the solver problem data, but differ in the inputs they accept and in how they compute the coefficients. For example, `CasADi` accepts vectorized expressions as input, but differentiates them through matrix–vector products, so the structure of the derivatives is never materialized and must be detected at runtime, over many sweeps of the expression graph. `AMPL` and `JuMP` let the user declare vectorized expressions but scalarize them when processing the data into coefficients. On the other hand, the canonicalization backends of `CVXPY` keep the expressions vectorized and need no detection step (for this reason, we only compare against these backends in Chapter 4).

2.1 Matrix stuffing in `CVXPY`

Matrix stuffing converts the conic-form expression trees into the problem data. After canonicalization every expression is affine, so the Jacobian of each node is a constant matrix, which matrix calculus [61] gives in closed form. Computing the problem data reduces to composing these constant Jacobians over the tree. This section reviews the derivative rules along with two design choices of the `CVXPY` implementation: the storage of the coefficients during the pass, and the caching of the problem data across parameter changes. Finally, we describe `CVXPY`'s current backends and their limitations.

Matrix calculus extends differentiation to functions whose arguments and values are vectors and matrices. Systematic rules for such derivatives date back to the 1940s [62]. The simplest convention is to vectorize [61]: every expression is identified with

$$\mathbf{vec}(\mathbf{X}) = \begin{bmatrix} x_{11} & \cdots & x_{m1} & x_{12} & \cdots & x_{mn} \end{bmatrix}^\top, \tag{2.1}$$

the vector that stacks the columns of $\mathbf{X} \in \mathbb{R}^{m \times n}$, and the derivative of one vectorized expression with respect to another is an ordinary Jacobian matrix. Under this convention, matrix operations become linear operators on the vectorized arguments, and their Jacobians are structured. For example, the matrix product $\mathbf{FX}$ has the Kronecker-structured, block-diagonal Jacobian $\mathbf{I} \otimes \mathbf{F}$ (Figure 2.1).

$$\texttt{X = cp.Variable((2, 2))} \quad \Longrightarrow \quad \frac{\partial\, \mathbf{vec}(\mathbf{X})}{\partial\, \mathbf{vec}(\mathbf{X})} = \begin{array}{c} \\ x_{11} \\ x_{21} \\ x_{12} \\ x_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

(a) A matrix variable is the identity map of its own entries.

$$\texttt{Y = F @ X} \quad \Longrightarrow \quad \frac{\partial\, \mathbf{vec}(\mathbf{Y})}{\partial\, \mathbf{vec}(\mathbf{X})} = \mathbf{I} \otimes \mathbf{F} = \begin{array}{c} \\ y_{11} \\ y_{21} \\ y_{12} \\ y_{22} \end{array} \begin{array}{c} \begin{array}{cccc} x_{11} & x_{21} & x_{12} & x_{22} \end{array} \\ \begin{bmatrix} f_{11} & f_{12} & 0 & 0 \\ f_{21} & f_{22} & 0 & 0 \\ 0 & 0 & f_{11} & f_{12} \\ 0 & 0 & f_{21} & f_{22} \end{bmatrix} \end{array}$$

(b) The Jacobian of the product $\mathbf{Y} = \mathbf{FX}$ is block-diagonal, one copy of $\mathbf{F}$ per column of $\mathbf{X}$.

Figure 2.1 Jacobians under the vectorization (2.1), for a matrix variable $\mathbf{X}$ and the product $\mathbf{FX}$, with $\mathbf{X}, \mathbf{F} \in \mathbb{R}^{2\times 2}$. The entries of $\mathbf{vec}(\mathbf{X})$, in column-major order, index the columns of the Jacobians.

2.1.1 Composing linear operators

Every node of a conic-form tree, not only the root, computes an affine function of the problem variables and can be written in the vectorized form

$$f(\mathbf{z}) = \mathbf{A}_f\, \mathbf{z} + \mathbf{b}_f = \begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} \begin{bmatrix} \mathbf{z} \\ 1 \end{bmatrix}, \tag{2.2}$$

where $\mathbf{z}$ is the stacked vector of all the problem variables, $\mathbf{A}_f$ is the Jacobian of f with respect to $\mathbf{z}$, and $\mathbf{b}_f$ is a constant vector. Stacking the augmented matrices of the constraint trees yields the problem data $[\mathbf{A}\ \mathbf{b}]$ of (1.2).

Matrix stuffing computes the augmented matrix in one bottom-up pass over the tree. A variable leaf contributes an identity block and a constant leaf contributes its value to the last column (Figure 2.3). An interior node with a single argument applies a constant linear operator $\mathbf{L}$ to the argument, which has the affine form (2.2) with augmented matrix $[\mathbf{A}_1\ \mathbf{b}_1]$; applying $\mathbf{L}$ gives

$$\begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} = \mathbf{L} \begin{bmatrix} \mathbf{A}_1 & \mathbf{b}_1 \end{bmatrix}. \tag{2.3}$$

The operator $\mathbf{L}$ is the Jacobian of the node with respect to its argument, so (2.3) is exactly the chain rule of matrix calculus. A node with k arguments applies one operator per argument,

$f = \sum_{i=1}^{k} \mathbf{L}_i f_i$, which gives

$$\begin{bmatrix} \mathbf{A}_f & \mathbf{b}_f \end{bmatrix} = \sum_{i=1}^{k} \mathbf{L}_i \begin{bmatrix} \mathbf{A}_i & \mathbf{b}_i \end{bmatrix}, \tag{2.4}$$

so the node costs one matrix product per argument.

In `CVXPY`, the linear operators divide into views and copies [63]. A view performs a simple manipulation of the coefficients of its argument. Atoms with one constant argument, such as `matmul` and `multiply`, make copies instead. The operator is a constant matrix built from the constant argument, and its product with the augmented matrix of the other argument is stored as a new coefficient matrix. Table 2.1 classifies a subset of the operators.

Table 2.1 A subset of the linear operators, classified as views or copies, applied to an argument f_1 with augmented matrix $[\mathbf{A}_1 \ \mathbf{b}_1]$, where $\otimes$ denotes the Kronecker product, $\odot$ the elementwise product, / the elementwise division, $\mathcal{I}$ an index set, and $\mathbf{diag}(\mathbf{v})$ the diagonal matrix with the entries of $\mathbf{v}$ on its diagonal.

Atom	**Operator L**	**Action on $[\mathbf{A}_1 \ \mathbf{b}_1]$**
Views		
$-f_1$	$-\mathbf{I}$	flip the sign of every stored entry
$f_1^\top$	$\mathbf{K}$, the vec-permutation	permute the rows
$f_1[\mathcal{I}]$	$\mathbf{P}_{\mathcal{I}}$, a row selection	keep the rows indexed by $\mathcal{I}$
reshape(f_1)	$\mathbf{I}$	none, the vectorization is unchanged
Copies		
$\mathbf{F} f_1$	$\mathbf{I} \otimes \mathbf{F}$	multiply by a block diagonal formed from $\mathbf{F}$
$f_1 \mathbf{B}$	$\mathbf{B}^\top \otimes \mathbf{I}$	multiply by a Kronecker matrix formed from $\mathbf{B}$
$\mathbf{C} \odot f_1$	$\mathbf{diag}(\mathbf{vec}(\mathbf{C}))$	multiply by a diagonal formed from $\mathbf{C}$
$f_1 / \mathbf{C}$	$\mathbf{diag}(\mathbf{vec}(\mathbf{C}))^{-1}$	multiply by the reciprocal diagonal

2.1.2 Example

We continue the example of §1.2.3, which canonicalized problem (1.5) into the conic-form trees of Figure 1.3. We now compute the problem data of these trees. The variables are first stacked into $\mathbf{z} = (t_1, t_2, \mathbf{x})$. Each variable then occupies a contiguous block of columns of $\mathbf{A}_f$, starting at its column offset within $\mathbf{z}$. The columns of the variables that do not appear in the expression f are zero, because f does not depend on their entries. Figure 2.2 gives an example of column offsets on another set of variables.

Matrix stuffing then runs bottom-up over the objective and the three constraint trees (Figure 2.3). Each leaf of $\mathbf{x}$ contributes an identity block of size n at the column offset of $\mathbf{x}$, and the constant leaf $\mathbf{g}$ contributes its value to the last column. The interior nodes apply their

$$\mathbf{z}^\top = \begin{array}{c} \begin{array}{ccccccc} \multicolumn{2}{c}{\mathbf{u}} & & \multicolumn{3}{c}{\mathbf{v}} & & \mathbf{w} \\ 0 & & & 2 & & & 5 & \\ \end{array} \\ \begin{bmatrix} u_1 & u_2 & v_1 & v_2 & v_3 & w_1 & w_2 \end{bmatrix} \end{array}$$

Figure 2.2 The column offsets of a stack of three variables $\mathbf{z} = (\mathbf{u}, \mathbf{v}, \mathbf{w})$ with $\mathbf{u} \in \mathbb{R}^2$, $\mathbf{v} \in \mathbb{R}^3$, and $\mathbf{w} \in \mathbb{R}^2$. Each offset is the total dimension of the preceding variables: $\mathbf{u}$ starts at column 0, $\mathbf{v}$ at 2, and $\mathbf{w}$ at 5. The entries of $\mathbf{z}$ index the columns of every augmented matrix $[\mathbf{A}_f \ \mathbf{b}_f]$.

operators to the matrices of their children; the product node $\mathbf{Fx}$, for example, multiplies the identity block of its argument by $\mathbf{F}$. Finally, the root stacks the augmented matrices of its children at their constraint row offsets into $[\mathbf{A}\ \mathbf{b}]$.

The augmented matrices of (2.2) can be stored in two ways (Figure 2.4). In the global layout, drawn in Figure 2.3, each node stores one matrix whose columns span the whole stacked vector $\mathbf{z}$, with the column offsets applied from the start. In the local layout, which `CVXPY` uses [63], each node keeps one coefficient block per variable in its subtree, and one for the constant data. The offsets are applied only after the pass, when each block is placed at its variable's columns and the blocks are concatenated into the full matrix $[\mathbf{A}\ \mathbf{b}]$. The local layout stores only the columns of the variables present in a subtree; the tradeoff is that each interior node applies its operator once per stored block, rather than once per argument.

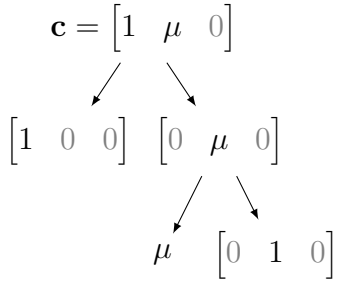
2.1.3 Parameter caching

In a problem family the structure is fixed and only the numeric parameter values change between solves. Under additional rules on how the parameters enter the expressions, known as Disciplined Parameterized Programming (DPP), the problem data is affine in the parameters [26]. To keep track of the parameter dependence, `CVXPY` adds a third dimension to the problem data, indexed by the parameter entries. Every coefficient matrix throughout the bottom-up pass becomes a tensor. At the root nodes we thus get the cached parameter maps

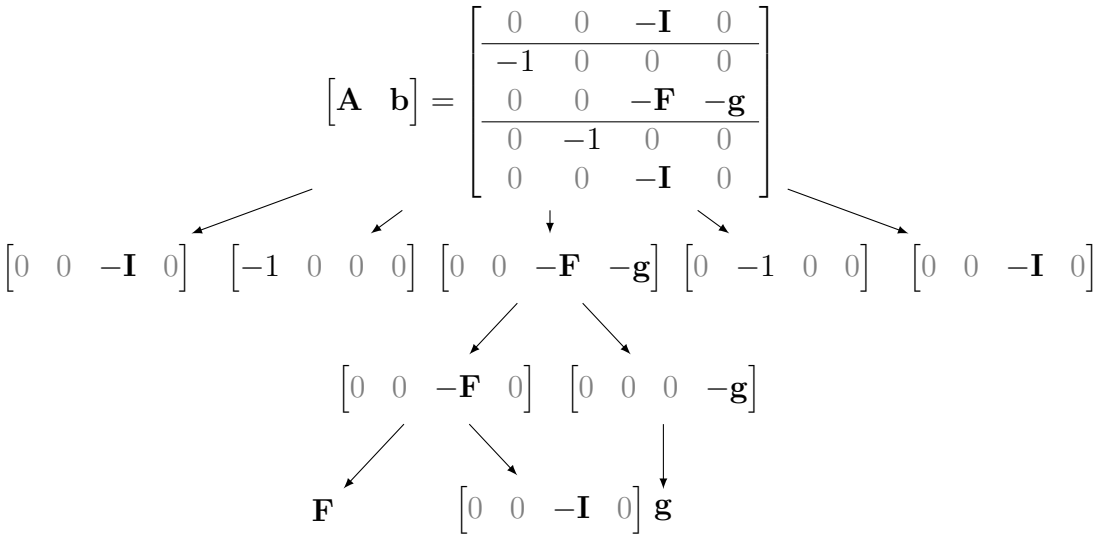
$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \mathbf{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}, \tag{2.5}$$

where $\mathbf{Q} \in \mathbb{R}^{n \times (p+1)}$ and $\mathbf{R} \in \mathbb{R}^{m(n+1) \times (p+1)}$. The objective map $\mathbf{Q}$ is shown in Figure 2.5 for problem (1.5). After a parameter change, a re-solve costs one sparse matrix–vector product per map.

Problems that do not conform to the DPP grammar, or that opt out of it with `ignore_dpp=True`, take a different approach to matrix stuffing. The reduction `EvalParams` replaces each pa-



(a) Matrix stuffing of the objective



(b) Matrix stuffing tree of the constraints

Figure 2.3 Matrix stuffing of problem (1.5) by a bottom-up pass over the conic-form expression trees of Figure 1.3: each node's matrix is obtained by applying a linear operation to the matrices of its children. The coefficients appear negated because (1.2) writes the constraints as $\mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}$. The horizontal rules in $[\mathbf{A}\ \mathbf{b}]$ separate the rows of the three constraints.

$$\left\{ \begin{array}{cc} \mathbf{x} & 1 \\ \mathbf{x} \mapsto [-\mathbf{F}], & -1 \mapsto [-\mathbf{g}] \end{array} \right\}$$

(a) Local layout.

$$\begin{array}{cccc} t_1 & t_2 & \mathbf{x} & 1 \\ [0 & 0 & -\mathbf{F} & -\mathbf{g}] \end{array}$$

(b) Global layout.

Figure 2.4 What each layout stores during the pass, at the node $-(\mathbf{F}\mathbf{x} - \mathbf{g})$ of problem (1.5): a dictionary of narrow per-variable blocks, or a single matrix spanning all columns.

$$\mathbf{Q} = \begin{array}{c} \\ t_1 \\ t_2 \\ \mathbf{x} \end{array} \begin{array}{c} \mathbf{g}^\top \\ \begin{bmatrix} 0 \cdots 0 \\ 0 \cdots 0 \\ 0 \cdots 0 \end{bmatrix} \end{array} \begin{array}{c} \mu \\ \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \end{array} \begin{array}{c} 1 \\ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \end{array} \qquad \mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}} = \begin{bmatrix} 1 \\ \mu \\ 0 \end{bmatrix}$$

Figure 2.5 The DPP path on the objective data of problem (1.5): the parameter map $\mathbf{Q}$ is built and cached once, and each re-solve recovers $\mathbf{c}$ with the single sparse product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$.

rameter leaf by a constant holding its current value, and the resulting parameter-free tree goes through the pass as if the problem had no parameters (Figure 2.6). The result is the same objective data $\mathbf{c}$ as in Figure 2.5, now obtained by repeating the whole matrix stuffing instead of applying the cached product $\mathbf{Q}\tilde{\boldsymbol{\theta}}$. The `ignore_dpp` path is more general, as it supports arbitrary functions of the parameters.

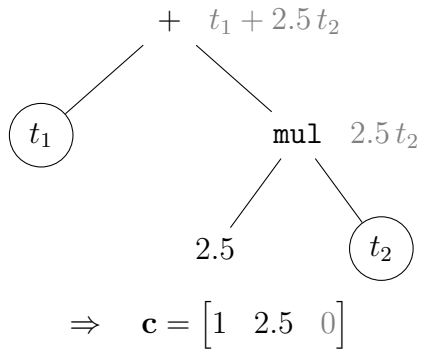


Figure 2.6 The `ignore_dpp` path on the objective of problem (1.5): the parameter leaf μ is replaced by a constant holding its current value 2.5, and the pass is repeated on the parameter-free tree on every change of the parameters.

`CVXPY` implements the pass in three canonicalization backends that differ in how they represent the parameter axis [64]:

- the C++ backend `cvxcore` keeps one sparse matrix per parameter slice and loops over the slices at every operator;
- the `SciPy` backend stacks the slices vertically into a single strided two-dimensional sparse matrix;
- the `COO` backend does not form the tensor explicitly, and instead stores each nonzero with an explicit parameter index alongside its row and column.

When canonicalization remains too slow, `CVXPYgen` can compile the problem ahead of time

[65]. It emits problem-specific C code that embeds the cached parameter maps (2.5) of a DPP problem.

The pass of this section produces the affine data $\mathbf{c}$ and $[\mathbf{A}\ \mathbf{b}]$. A quadratic objective is left by the frontend as a sum of quadratic forms with constant matrices (§1.2.2), and a second matrix stuffing pass assembles these matrices into the diagonal blocks of the matrix $\mathbf{P}$ of (1.2) [63].

All three backends share two limitations. Firstly, they delegate their matrix operations to a general-purpose sparse library, Eigen [59] in `cvxcore` and `scipy.sparse` [57] in the Python backends. The storage formats and kernels are fixed by the library, so structure known at canonicalization time cannot be exploited. In `cvxcore` the views of Table 2.1 also materialize explicit sparse operators. For example, a reshape builds a sparse identity matrix and multiplies the coefficients by it.

Secondly, the canonicalization backends store dense constant matrices as sparse matrices. When dense blocks appear in the problem data, they are both stored in an inconvenient format and computed without the optimized BLAS kernels of §1.4.1. Our approach in Chapter 3 addresses both limitations.

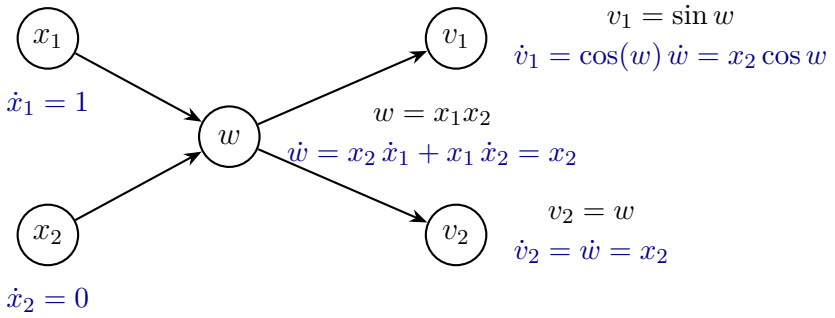
2.2 Sparse derivatives from detected structure

This section reviews methods that compute the full sparse Jacobian of a vector-valued function, given as an arbitrary program in a general-purpose language. No modelling language declares the structure of the program, so the sparsity of the derivatives must be detected [66]. Modern surveys of AD address dense derivatives, or the gradient of a scalar function, which reverse mode computes in a single sweep over the computational graph [40, 67]. In those settings, AD refers to the evaluation of matrix–vector products with the Jacobian or the Hessian, and the explicit matrix is never formed. Factorizations, direct linear solves, and many optimization solvers instead require the explicit matrix [66]. The methods of this section recover the full sparse matrix from multiple matrix–vector products. To reduce the number of products, graph colouring groups the vectors that can be propagated together, so that one product recovers several columns at once.

2.2.1 Matrix-vector products and accumulation

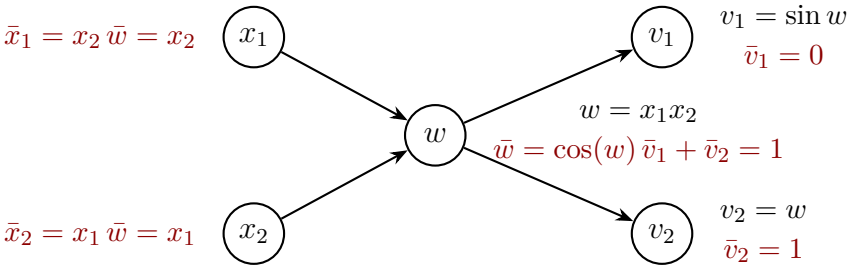
AD computes one matrix–vector product with the derivative per sweep of the computational graph, a traversal that applies the chain rule node by node at about the cost of one function evaluation [46]. Forward mode starts from a seed vector $\mathbf{v}$ at the inputs and propagates

tangents, and the Jacobian-Vector Product (JVP) $\mathbf{Jv}$ appears at the outputs. Reverse mode starts from a seed $\mathbf{w}$ at the outputs and propagates adjoints, and the Vector-Jacobian Product (VJP) $\mathbf{J}^\top\mathbf{w}$ appears at the inputs (Figures 2.7 and 2.8). For the Hessian, the analogous Hessian-vector product $\mathbf{Hv}$ comes from one forward sweep over one reverse sweep.



$$\begin{bmatrix}\dot{v}_1\\ \dot{v}_2\end{bmatrix} = \begin{bmatrix}x_2\cos w\\ x_2\end{bmatrix} = \mathbf{J}\mathbf{e}_1, \qquad \mathbf{J} = \begin{bmatrix}x_2\cos w & x_1\cos w\\ x_2 & x_1\end{bmatrix}.$$

Figure 2.7 Forward sweep of the computational graph of $F(x_1, x_2) = (\sin(x_1x_2),\ x_1x_2)$, with the intermediate value $w = x_1x_2$. Alongside its value, each node carries one tangent $\dot{v}$ (in blue), updated by the chain rule following the order of evaluation. The seed $(\dot{x}_1, \dot{x}_2) = (1, 0)$ delivers the first column of $\mathbf{J}$ at the outputs.

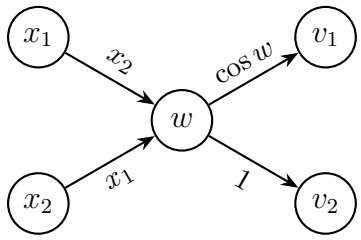


$$\begin{bmatrix}\bar{x}_1 & \bar{x}_2\end{bmatrix} = \begin{bmatrix}x_2 & x_1\end{bmatrix} = \mathbf{e}_2^\top\mathbf{J}.$$

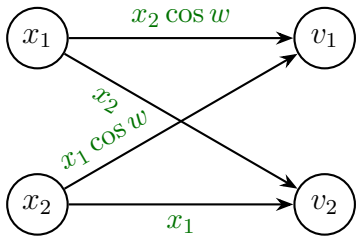
Figure 2.8 Reverse sweep of the computational graph of Figure 2.7. Each node carries one adjoint $\bar{v}$ (in red), updated by the chain rule against the order of evaluation. The seed $(\bar{v}_1, \bar{v}_2) = (0, 1)$ delivers the second row of $\mathbf{J}$ at the inputs.

Recovering the full matrix takes a sequence of sweeps, one per seed vector. The n unit seed vectors recover $\mathbf{J}$ column by column in forward mode, and the m unit seed vectors recover it row by row in reverse mode [46]. The other approach is accumulation, which combines the local derivatives of the graph directly into $\mathbf{J}$ by eliminating one intermediate vertex at a time

(Figure 2.9) [68]; finding the optimal elimination order is NP-complete [46, 69]. Batching the n unit seeds into one forward sweep gives each node a whole tangent, the derivative of that node with respect to all the inputs. Eliminating the vertices in the order of evaluation stores the same derivative at each node, so the batched sweep and forward-order elimination perform the same computation.



(a) Each edge carries a local derivative.



(b) After eliminating w, the edges are the entries of $\mathbf{J}$.

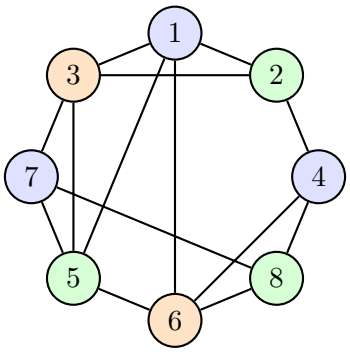
Figure 2.9 Accumulation of the Jacobian by vertex elimination, on the graph of Figure 2.7. Eliminating the intermediate vertex w connects each of its predecessors to each of its successors and multiplies the local derivatives along the path (in green); parallel edges, when they arise, are summed. Once every intermediate vertex is eliminated, the graph is bipartite and its edges are the entries of $\mathbf{J}$.

In practice, AD tools such as ADOL-C [70] and CppAD [71] record the scalar graph as a runtime trace, one record per elementary operation [46, 72]. A single matrix–vector product unrolls into one scalar multiplication for every entry of the matrix, together with the additions that sum them, so the trace of a realistic problem can sometimes reach millions of nodes. On such a trace, every intermediate vertex would store its derivative with respect to all the inputs, so accumulation is rarely used in practice. When $\mathbf{J}$ is sparse, most of each sweep is then wasted on zeros, and the cost of recovering the matrix scales with its dimension rather than with its number of nonzeros.

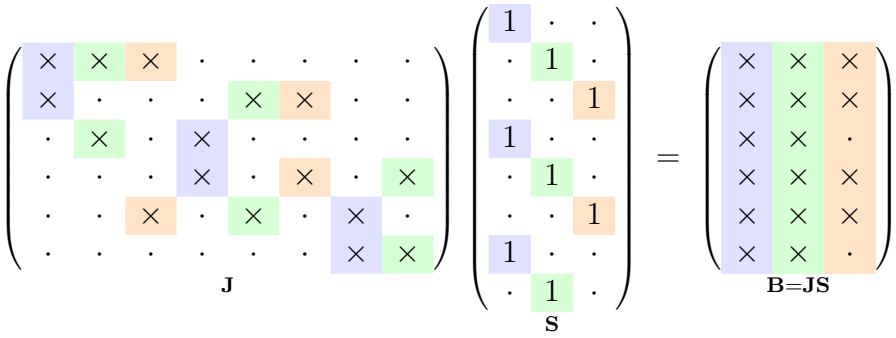
2.2.2 Compression by graph colouring

Several columns of a sparse Jacobian can share one seed [28]. Two columns with no nonzero in a common row are structurally orthogonal, and one sweep recovers both, each entry landing in a distinct row of the product. Finding the fewest such groups was later formalized as a colouring of the column intersection graph, which has one vertex per column and an edge between two columns that share a row [29, 73]. Figure 2.10 colours the graph of a 6×8 Jacobian and recovers the matrix in three sweeps instead of eight. The approach extends to the Hessian, which compresses further because its symmetry allows dedicated star and

acyclic colourings [74, 75].



(a) The column intersection graph of **J**, with one vertex per column and an edge between two columns that share a row, coloured with three colours.



(b) The seed matrix **S** sums the unit vectors of each colour class, and the compressed product $\mathbf{B} = \mathbf{JS}$ recovers every nonzero of **J**, each in a distinct row of its compressed column.

Figure 2.10 Graph colouring compresses the seeds. Colouring the column intersection graph groups the columns of the sparse Jacobian **J** into three structurally orthogonal classes, and the matrix is recovered in three sweeps instead of eight.

The sparse Jacobian is obtained from the products in three stages: detect the sparsity pattern, compress it by colouring, and propagate the compressed seeds of Figure 2.10 through the graph to recover the nonzero entries [66]. Detection propagates sparsity patterns instead of values through the same graph, replacing each arithmetic operation by a union of index sets [46]. The pattern and its colouring are computed once and reused, so their cost is amortized across repeated evaluations of the same function.

2.2.3 CasADi

CasADi [12] is an open-source framework for nonlinear optimization and optimal control built around AD. It is widely used in academia and industry, notably for model predictive control, trajectory optimization, robotics, and energy systems [76]. Its core is written in C++ and is used from Python, MATLAB, or Octave. It provides the building blocks of gradient-based optimization: symbolic expressions, derivatives, integrators, and interfaces to NLP and QP solvers. The gradients, Jacobians, and Hessians these solvers require are obtained by differentiating the model.

CasADi supports two types of expressions, and both return the matrix derivatives in a sparse format, which is expected by most solvers [76]. An SX graph is scalar, with exact node-by-node sparsity, but its size grows with the problem data. An MX graph is vectorized. Its nodes are whole matrix operations, so the graph stays small even when the model is large. Either graph can be exported as self-contained C code and compiled ahead of time for embedded applications.

CasADi computes its derivatives by the detection, colouring, and batched sweeps of this section, and caches the pattern, the colouring, and the compiled derivative function across evaluations. The following listing rebuilds the conic-form trees of Figure 1.3, obtained after canonicalization, as MX expressions; differentiating them recovers the problem data that matrix stuffing assembles in §2.1.2.

```
import casadi as cs
t1, t2 = cs.MX.sym("t1"), cs.MX.sym("t2")
x = cs.MX.sym("x", n)
g, mu = cs.MX.sym("g", m), cs.MX.sym("mu")
z = cs.vertcat(t1, t2, x)               # stacked variables
expr = cs.vertcat(x, t1, F @ x - g, t2, x) # stacked affine expressions
A = -cs.jacobian(expr, z)               # the matrix A
b = cs.substitute(expr, z, 0)           # the vector b
c = cs.gradient(t1 + mu * t2, z)        # the objective vector c
```

The Jacobian with respect to the stacked variable $\mathbf{z}$ recovers the matrix $\mathbf{A}$ of Figure 2.3 entry by entry; differentiation is the equivalent of matrix stuffing, with the coefficients obtained by those sweeps rather than by a bottom-up pass over the tree. The parameters remain symbols, so $\mathbf{b}$ and $\mathbf{c}$ are expressions of $\mathbf{g}$ and μ, re-evaluated for each parameter value. This is how the qpsol interface builds its problem data internally, for problems with a quadratic objective and affine constraints. A model written at the same high level as in CVXPY, with the norms of (1.5) kept in the expression graph, is solved through nlpsol instead. The same

sweeps assemble the gradient, constraint Jacobian, and Hessian of the Lagrangian that the NLP solver queries at each iteration.

2.3 Coefficient extraction from declared structure

In contrast to §2.2, the methods of this section require the user to declare the structure of the problem. In `AMPL` and `JuMP`, the user writes each objective and constraint as a sum of terms, and the coefficient of every linear and quadratic term is stored as it is typed. Neither language has an atom library or a canonicalization step, so a nonlinear expression such as a norm is left to the solver (`AMPL`) or written in conic form by the user (`JuMP`). In place of differentiation, `AMPL` writes the stored coefficients to a `.nl` file that the solver parses, and `JuMP` keeps them as typed in-memory term lists that are copied to the solver.

2.3.1 `AMPL`

`AMPL` [5] is a commercial algebraic modelling language for large-scale mathematical programming, developed at Bell Laboratories in the 1980s. It is one of the most widely used modelling languages in academia and industry, applied to production planning, scheduling, logistics, and energy systems. A model is written declaratively, close to its mathematical statement, from sets, indexed parameters, indexed variables, and algebraic expressions over them. Data is supplied separately, so one model can be used for many problem instances. Models are written in the `AMPL` language itself or built through its Python interface [77], and reach a large collection of commercial and open-source solvers through a single instance format.

The translator runs as its own process. It parses the model and the data, instantiates every constraint, and writes the resulting instance to a `.nl` file [30]. Each solver is reached through a driver, a separate executable that reads the `.nl` file and calls the solver. A driver is built on one of two libraries that supply the reader and the expression handling. The AMPL Solver Library (ASL) is the original, a C library that parses the `.nl` format and evaluates the expressions it holds [31]. `MP` is the newer one, which AMPL supplies for solvers that need no derivatives [78]. This division between a translator that writes an instance and a driver that reads it still defines how most solvers receive nonlinear problems.

The `.nl` file is an instance format rather than a model format. Sets and indexing are resolved by the translator, and what remains is one flat record per objective and constraint. An indexed declaration such as `var x {1..n}` becomes n scalar variables, and an indexed constraint becomes one record per index, so the file has no notion of a vector. The linear part of each record is an explicit list of (variable, coefficient) pairs, stored alongside the bounds

and the nonzero counts of the instance. Only the nonlinear parts are kept as expression graphs, written in prefix notation [30].

The following model states problem (1.5) in `AMPL`. The language has no norm atom and no canonicalization step, so the norms are entered as square roots of sums of squares. What the solver receives depends on its driver. An `MP`-based driver rewrites the two norms into second-order cones for a conic solver, and an ASL-based driver passes on a general nonlinear program.

```
param m; param n;
param F {1..m, 1..n};
param g {1..m};
param mu >= 0;
var x {1..n} >= 0;
minimize obj:
    sqrt(sum {i in 1..m} (sum {j in 1..n} F[i,j]*x[j] - g[i])^2)
  + mu * sqrt(sum {j in 1..n} x[j]^2);
```

For an instance with $m = 3$, $n = 2$, and $\mu = 0.1$, the translator writes the following `.nl` file, shown here in abbreviated form. The comments are added by hand to make the records readable and are not written by the translator.

```
g3 1 1 0    # text format; 9 more header lines follow
 2 0 1 0 0  # 2 vars, 0 constraints, 1 objective, 0 ranges, 0 eqns
...
O0 0        # objective 0, an expression graph in prefix notation
o0          #   add
o39         #     sqrt
o54         #       sum of 3 terms, one per row of F
...         #         affine subtrees F[i,1]*x[1] + F[i,2]*x[2] - g[i]
o2          #   multiply
n0.1        #     the value of mu
...         #     sqrt subtree of the second norm
b           # variable bounds: x[1] >= 0, x[2] >= 0
2 0
2 0
```

The whole objective is written as one expression graph, the constraint $\mathbf{x} \geq 0$ reduces to the two bound records of the `b` segment, and the value of μ is folded into the graph as the constant `n0.1`, so a parameter change requires a new file. For a linear or quadratic program, no graph remains and the file contains the complete problem data as coefficient lists.

Inside the driver, the bounds and the linear coefficient lists are read directly from the file, at a cost linear in the number of nonzeros, and only the nonlinear parts are rebuilt as expression graphs. What happens to those graphs depends on the library. `MP` matches second-order and exponential cones in the rebuilt graphs. A conic solver such as Gurobi or MOSEK then receives cone constraints rather than the square roots the model states. ASL instead differentiates the graphs by reverse-mode AD [79], with sparse Hessians assembled from partial separability [80].

Any system that writes `.nl` reaches every solver that reads it. The format has grown since, with extensions such as logical constraints, and AMPL omits the added header fields when a model does not use them, so older drivers still read the files it writes [30]. An instance is an ordinary file, so it can be archived, shipped to reproduce a bug, and rerun on another machine. However, serializing the instance to disk can be slow, and every solve repeats it, rebuilding any expression graphs from scratch.

2.3.2 `JuMP`

`JuMP` [11, 81] is an open-source algebraic modelling language embedded in Julia, developed at MIT and first released in 2013. It is the most widely used modelling language in Julia, applied in academia and industry to power systems, portfolio optimization, scheduling, and routing. A model is written with the macros `@variable`, `@constraint`, and `@objective`, and the same model can be passed to any of more than fifty supported solvers [82]. The abstraction layer between models and solvers is MOI [32].

The macros of `JuMP` rewrite each expression as it is entered. `@objective` and `@constraint` build affine and quadratic expression objects term by term, so the model stores coefficient lists instead of an expression tree. These lists are typed by MOI. A `VectorAffineFunction` is a list of (row, coefficient, variable) terms plus a constant vector. A `ScalarQuadraticFunction` adds a list of (coefficient, variable, variable) triples. A constraint states that one such function lies in a set. For a nonnegative orthant or a second-order cone, this is exactly the functional conic form of §1.2.1. MOI also supports other sets, such as integrality, special ordered sets, and indicator constraints [83].

The following model states problem (1.5) in `JuMP`. The user declares the epigraph variables and cone memberships of (1.6) directly.

```
using JuMP
model = Model()
@variable(model, x[1:n] >= 0)
@variable(model, t1); @variable(model, t2)
```

```
@constraint(model, [t1; F * x - g] in SecondOrderCone())
@constraint(model, [t2; x] in SecondOrderCone())
@objective(model, Min, t1 + mu * t2)
```

The first cone constraint is stored as

```
VectorAffineFunction([(1, 1.0, t1), (2, F[1,1], x[1]), ...],
                     [0.0, -g[1], ..., -g[m]]) in  SecondOrderCone(m + 1)
```

and the expression itself is not retained after the macro returns.

When the model is passed to a solver, MOI copies each constraint into the solver's own representation with `copy_to`. Solvers that want matrix data commonly implement that copy through the `MatrixOfConstraints` utility, which concatenates the stored term lists into one sparse matrix in the solver's format [83]. Coefficient extraction passes once over the stored terms and forms the sparse matrix, with no traversal of an expression graph and no differentiation.

When a solver does not support a constraint type natively, MOI reformulates it through its bridge system. A bridge is a structural rewrite from one function-set pair to another, for instance from a rotated to a standard second-order cone. MOI selects and composes them automatically [32]. Bridging manipulates stored coefficients and adds auxiliary variables and constraints.

`JuMP` also has a nonlinear path, in which expression graphs are retained and derivatives are computed by sparse AD. The MOI representation is general, and a new problem class enters as a new function or set rather than as a change to the interface. However, vectorized expressions written in the model are stored as scalar coefficients, and each is expanded into one term per nonzero as the macro runs.

CHAPTER 3 A LIGHTWEIGHT CANONICALIZATION ENGINE

Mathematically, `SparseDiffEngine` computes the same derivatives as `CVXPY`'s canonicalization backends. Both apply the chain rule of matrix calculus to trees of vectorized expressions, and both produce the Jacobian and the Hessian defined in §1.3. The differences are in the implementation. Firstly, a custom sparse matrix data structure allows a natural split between the symbolic and numeric phases described in §1.1. This separation is particularly useful for compiling problems involving parameters, where the structure is fixed but the numerical values can change. Secondly, the data of most optimization problems is not only sparse but also structured, with dense blocks and diagonal patterns. In `SparseDiffEngine` we introduce two new matrix classes for this purpose, `permuted-dense` and `stacked-permuted-dense`, which let us use dense storage and BLAS subroutines when possible.

Finally, some important design decisions differ from `CVXPY`'s backends for simplicity rather than speed. Every node carries a single Jacobian whose columns span all of the variables of the problem, rather than a dictionary of local Jacobians keyed by variables. As a consequence, the chain rule is a single matrix product, and the constraint matrix is built by stacking the Jacobians of the constraints. In addition, quadratic functions have a dedicated Hessian pass, instead of the variable substitution and second matrix stuffing pass of §2.1.

The rest of the chapter is organized as follows. We begin with a brief overview of the engine, describing how expressions are represented and how the code is organized (§3.1). We then present the matrix representations used for the derivative computations (§3.2), where we focus much of our attention on the `permuted-dense` matrix, which appears most often. Next we describe the algorithms, or kernels, which operate on and combine these matrix representations (§3.3). Finally, we show how `SparseDiffEngine` is used within `CVXPY` to obtain the problem data (§3.4).

3.1 Overview of the engine

`SparseDiffEngine` is a small C library which is around seventeen thousand lines of code (and a similar number of lines for the test suite). The expression node is defined in `expr.h`; atoms that carry extra data, such as the exponent of a power, define a larger structure in `subexpr.h`. Each atom is implemented in its own file in the `atoms/` directory; there are around forty-five of them, divided into six subdirectories based on their mathematical properties. The matrix data structures and the computational kernels that operate on them are in `utils/`, and

`problem.c` collects the objective and constraints of a problem and evaluates the values and derivatives that a solver asks for.

3.1.1 Atom classes

Atoms in the same subdirectory compute their derivatives in the same way (Table 3.1). Canonicalization leaves the constraints affine and the objective affine or quadratic. The quadratic atoms of a DCP problem (a squared power, a quadratic over a linear term, a Huber penalty) are all rewritten as quadratic forms by the canonicalization frontend (§1.2.2), so only affine atoms and quadratic forms reach the engine on this path. This chapter describes the affine atoms; the quadratic form contributes the objective matrix $\mathbf{P}$ through the Hessian (§3.4). The remaining classes are used by the nonlinear programming (NLP) interface of `CVXPY` [34]. Each atom implements three operations: a forward pass, which evaluates the

Table 3.1 The atom classes and how each computes its Jacobian.

Class	Examples	Jacobian computation
affine	`sum`, `index`, `left_matmul`	fixed; index maps and copies
elementwise, full domain	`exp`, `power`, `logistic`	diagonal scaling of the child Jacobian
elementwise, restricted domain	`log`, `entr`, `atanh`	diagonal at the variable's columns
bivariate, full domain	`multiply`, `matmul`	two-term chain rule
bivariate, restricted domain	`quad_over_lin`, `rel_entr`	two-term chain rule
other	`quad_form`, `prod`	specific to each atom

atom at a point; a symbolic pass, which computes the sparsity patterns of its derivatives and allocates memory for them; and a numeric pass, which computes the numerical values inside the fixed sparsity patterns. The constructor of the atom stores pointers to these three functions in the expression node.

3.1.2 The expression node

An expression node stores its dimensions, pointers to at most two children, a buffer for its value, and its derivative matrices. Every derivative matrix has `n_vars` columns, the total number of entries of all the variables of the problem. A simplified version of the definition in `expr.h` is shown below.

```
typedef struct expr
```

```
{
    int d1, d2, size, n_vars, refcount, var_id;
    struct expr *left;
    struct expr *right;

    double *value;   /* forward value              */
    matrix *jacobian; /* Jacobian w.r.t. the variables */

    forward_fn      forward;
    jacobian_init_fn jacobian_init_impl; /* symbolic pass */
    eval_jacobian_fn eval_jacobian;    /* numeric pass */
} expr;
```

Trees are built bottom up, with one constructor function per atom; the Python bindings expose the same constructors, so a `CVXPY` expression is converted in one pass over the tree, which calls the constructor of the corresponding atom at each node (§3.4). Expressions that appear multiple times can be shared, in which case the tree becomes a directed acyclic graph. Nodes are therefore reference counted, and the symbolic pass returns early if a node was already initialized.

There are two leaf nodes, variables and parameters. A variable occupies a contiguous slice of the global variable vector, starting at its offset `var_id`; its Jacobian is a selection matrix with one unit entry per row, written once during the symbolic pass. Unlike `CVXPY`, there is no separate constant node: constants and parameters share a single leaf type, distinguished by whether the leaf can be updated. The Jacobian of this leaf is empty, because parameters are fixed during a solve.

3.2 Matrix representations

The Jacobian of a node is stored in whichever representation matches its structure. The default is a generic sparse matrix in the CSR format of §1.4.2, which makes no assumption about where the nonzeros are. Problem data is often dense in a small block, and the engine adds two representations that keep such blocks dense so that products on them go to level-3 BLAS. Each representation implements the same operations, so an atom picks one when it allocates its Jacobian during the symbolic pass, and every later operation is dispatched to the representations involved.

3.2.1 The permuted dense matrix

The Jacobians that appear during canonicalization are usually not sparse in the sense of having scattered, isolated nonzeros. Multiplication by problem data places the data itself, or a product involving it, as a dense block at a subset of the rows and columns of the Jacobian. Consider the expression $\mathbf{Bx}$ with dense data $\mathbf{B} \in \mathbb{R}^{50\times 50}$, in a problem with ten thousand variables. Its Jacobian with respect to the full variable vector is a matrix with ten thousand columns that contains $\mathbf{B}$ in the fifty columns of $\mathbf{x}$ and is zero everywhere else (Figure 3.1). In the generic sparse format that block costs a column index for each of its twenty-five hundred entries, and every later product involving it runs through the sparse loop instead of a BLAS call.

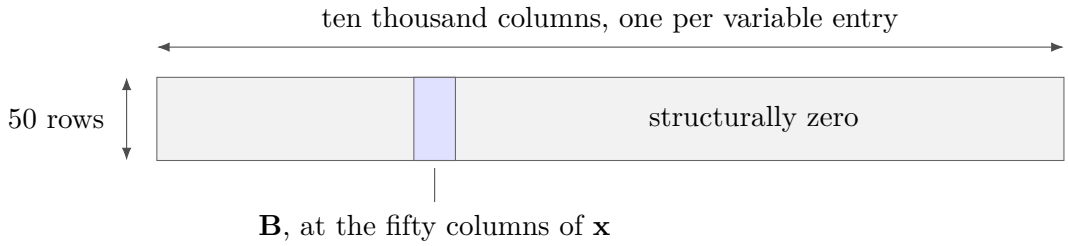


Figure 3.1 The structure of the Jacobian of $\mathbf{Bx}$ with respect to the full variable vector, for dense data $\mathbf{B} \in \mathbb{R}^{50\times 50}$ in a problem with ten thousand variables. The nonzeros are exactly the entries of $\mathbf{B}$, and the width of the matrix is set by the problem rather than by the expression.

Mixing dense blocks into sparse storage appears in several settings. Supernodal and multifrontal solvers gather dense submatrices of a sparse factor and process them with level-3 BLAS [84,85]; see also the survey [86]. Blocked sparse formats and autotuned kernels exploit small dense blocks found in the matrix [87, 88]. In these systems the blocks are discovered in the data; here they are known from canonicalization, which is what makes it possible to keep them in a dedicated representation from the start.

A *permuted dense* matrix stores such a Jacobian in *factored form*, as the dense block together with the index vectors that place it, and the global matrix is never assembled. It is a triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$, where $\mathbf{X} \in \mathbb{R}^{m_0\times n_0}$ is a dense block and $\mathbf{r}$ and $\mathbf{c}$ are strictly increasing index vectors, representing

$$\mathbf{A} = \mathbf{S}_r^\top \mathbf{X} \mathbf{S}_c \in \mathbb{R}^{m\times n}, \qquad A_{r_i, c_j} = X_{ij}, \tag{3.1}$$

where $\mathbf{S}_r \in \{0,1\}^{m_0\times m}$ and $\mathbf{S}_c \in \{0,1\}^{n_0\times n}$ are the selector matrices whose rows are the rows of the identity indexed by $\mathbf{r}$ and $\mathbf{c}$. Multiplying a vector by $\mathbf{S}_r$ selects the entries indexed by $\mathbf{r}$, and multiplying by $\mathbf{S}_r^\top$ scatters them back into a vector of length m, with zeros elsewhere.

All other entries of $\mathbf{A}$ are structurally zero and never stored. Figure 3.2 shows an example with a 3×3 block inside a 6×7 matrix, and Figure 3.3 writes out the two selectors of that example.

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{00} & \cdot & x_{01} & \cdot & \cdot & x_{02} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}$$

$$\mathbf{r} = (1, 3, 4), \qquad \mathbf{c} = (0, 2, 5).$$

Figure 3.2 A permuted dense matrix. The dense block $\mathbf{X}$ is stored once, contiguously; the index vectors $\mathbf{r}$ and $\mathbf{c}$ place its entries in the global index space, and every entry outside the shaded cells is structurally zero and never stored. Each block row keeps its own shade, reused in Figure 3.4.

$$\mathbf{A} = \underbrace{\begin{pmatrix} \cdot & \cdot & \cdot \\ 1 & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot \\ \cdot & \cdot & 1 \\ \cdot & \cdot & \cdot \end{pmatrix}}_{\mathbf{S}_r^\top} \underbrace{\begin{pmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} 1 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot \end{pmatrix}}_{\mathbf{S}_c}$$

Figure 3.3 The two selector matrices of the example of Figure 3.2, with $\mathbf{r} = (1, 3, 4)$ and $\mathbf{c} = (0, 2, 5)$. The rows of $\mathbf{S}_r$ are the rows of the 6×6 identity indexed by $\mathbf{r}$, and the rows of $\mathbf{S}_c$ are the rows of the 7×7 identity indexed by $\mathbf{c}$. Their product with $\mathbf{X}$ is the $\mathbf{A}$ of Figure 3.2; column i of $\mathbf{S}_r^\top$ carries block row i to global row r_i, so the shades match. Neither selector is ever formed.

The block $\mathbf{X}$ is stored contiguously in row-major order and the index vectors as two integer arrays, so the storage does not depend on the global dimensions m and n. Two inverse maps send a global row or column index to its position in the block, or to -1 when the index is not in the block, which makes a membership test constant time. Every kernel below determines the shape of its result from these integer arrays alone, and does its floating point work on the dense block.

Transposition and the structural atoms turn a permuted dense matrix into another permuted dense matrix. Transposition swaps the two index vectors and transposes the block. Indexing, broadcasting, and the row partitions of `sum` work the same way: because row r_i of $\mathbf{A}$ is row i of $\mathbf{X}$ and every other row is zero, an operation on the rows of $\mathbf{A}$ reduces to the same operation on the rows of $\mathbf{X}$, guided by the inverse row map.

output row i	0	1	2
requested global row s_i	3	0	4
inverse row map $\mathbf{r}^{-1}[s_i]$	1	-1	2
action	copy $\mathbf{X}_{1,:}$	zero row	copy $\mathbf{X}_{2,:}$

$$\mathbf{C} = \mathbf{A}[(3,0,4),\,:] \;=\; \begin{pmatrix} x_{10} & \cdot & x_{11} & \cdot & \cdot & x_{12} & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ x_{20} & \cdot & x_{21} & \cdot & \cdot & x_{22} & \cdot \end{pmatrix} \begin{matrix} \leftarrow \mathbf{A}_{3,:} \\ \leftarrow \mathbf{A}_{0,:} = 0 \\ \leftarrow \mathbf{A}_{4,:} \end{matrix}$$

$$\mathbf{X}_C \;=\; \mathbf{X}[(1,2),\,:] \;=\; \begin{pmatrix} x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{pmatrix}, \qquad \mathbf{r}_C = (0,2), \qquad \mathbf{c}_C = \mathbf{c} = (0,2,5).$$

Figure 3.4 Row indexing of the permuted dense matrix $\mathbf{A}$ of Figure 3.2. The table traces the scan of the inverse row map: each requested global row yields a block row of $\mathbf{X}$ to copy, or -1, in which case the output row is structurally zero and not stored. Shades follow the block rows of Figure 3.2. The result is the permuted dense triple $(\mathbf{X}_C, \mathbf{r}_C, \mathbf{c}_C)$.

Figure 3.4 traces the indexing of rows $(3,0,4)$ of the matrix in Figure 3.2. A repeated index in the request would copy the same block row twice. Rows outside $\mathbf{r}$ are requested whenever an earlier atom widened the row space: `diag` places the n rows of the Jacobian of its argument at the diagonal positions of a matrix with n^2 rows, so all but n of them are structurally zero, and an indexing or a sum applied downstream reads them through the inverse row map.

The product of two permuted dense matrices is again permuted dense. The Jacobian of `left_matmul` is the product $\mathbf{BA}$ of its data block with the Jacobian of its argument, and the Hessian of the quadratic form accumulates $\mathbf{B}^\top\mathbf{DA}$ with a diagonal weight $\mathbf{D}$. The transposed form factors as

$$\mathbf{B}^\top\mathbf{A} = \mathbf{S}_{c_B}^\top \mathbf{X}_B^\top \underbrace{\left(\mathbf{S}_{r_B}\mathbf{S}_{r_A}^\top\right)}_{\text{index-set intersection}} \mathbf{X}_A \mathbf{S}_{c_A}, \tag{3.2}$$

and $\mathbf{BA}$ factors the same way, with $\mathbf{c}_B$ in place of $\mathbf{r}_B$ in the middle factor and as the row vector of the result. The middle factor is a zero-one matrix whose entry (k,l) is one exactly when the two indices agree, so the sparsity analysis of the product amounts to intersecting two sorted integer arrays. The result has column vector $\mathbf{c}_A$ and a row vector that is known before any numerical values exist. For the values there are three cases. If the two index sets

are equal, the product is computed on the two blocks with a single call to `dgemm`, the double precision general matrix–matrix product of level-3 BLAS (§1.4.1). If they overlap partially, the matching rows are first gathered into a workspace (allocated during the symbolic pass), followed by the same `dgemm`. If they are disjoint, the product is zero, and nothing is computed in the numeric pass.

Two dense operands require a chain of products such as $\mathbf{A}(\mathbf{Bx})$, because the Jacobian of a variable leaf is a generic sparse selection matrix. On the Hessian side they require a quadratic form of a composite argument, $(\mathbf{Ax})^\top \mathbf{Q}\mathbf{Ax}$; a quadratic form of a bare variable has Hessian $2\mathbf{Q}$ and is written out with no product at all.

In the factored form (3.1), the numeric pass of each kernel is a few BLAS calls on the dense block. The product $\mathbf{A}^\top \mathbf{D}\mathbf{A}$, for example, is computed with one copy, m_0 scalings, and one `dgemm`, and the cost does not depend on the global dimensions.

The gain is limited by what the representation can hold. A permuted dense matrix has nonzeros only inside its block, so a Jacobian that mixes a dense block with scattered entries falls back to the default CSR representation. This happens on every addition. The Jacobian of $\mathbf{Ax} + \mathbf{y}$ carries the block of $\mathbf{A}$ at the columns of $\mathbf{x}$ and an identity at the columns of $\mathbf{y}$, which no single block covers, so the sum is performed on CSR views of its operands. The view itself is cheap, because its value array points into the dense block and it is built once during the symbolic pass, but its result is a generic sparse matrix, and the dense kernels no longer apply to it or to anything computed from it. The remaining risk is dense data that contains many zeros, which would be stored and multiplied densely, spending floating point work on zeros. Because `CVXPY` supplies sparse data in sparse formats, this does not arise in practice.

3.2.2 The stacked permuted dense matrix

Multiplying an $n \times p$ expression by data $\mathbf{A} \in \mathbb{R}^{m \times n}$ acts on the vectorized expression as the block diagonal operator of Table 2.1, one copy of $\mathbf{A}$ per column of the operand; this is the `left_matmul` atom. Rather than form that operator, the atom keeps a single copy of $\mathbf{A}$ and applies it block by block. In memory, the p segments of length n of a vector $\mathbf{x}$ form a $p \times n$ row-major matrix $\tilde{\mathbf{X}}$, and by a standard vectorization identity [61],

$$\mathbf{y} = (\mathbf{I}_p \otimes \mathbf{A})\,\mathbf{x} \quad \Longleftrightarrow \quad \tilde{\mathbf{Y}} = \tilde{\mathbf{X}}\mathbf{A}^\top, \tag{3.3}$$

so applying the operator to a vector is a single `dgemm` on the stored $\mathbf{A}$, and the storage does not grow with the number of blocks p.

Storing the Jacobian of the atom is a separate problem. The parent node receives $(\mathbf{I}_p \otimes \mathbf{A})\,\mathbf{J}$, where $\mathbf{J}$ is the Jacobian of the child, and transposes, indexes, and multiplies it like any other Jacobian. This matrix is made of p dense blocks at disjoint rows, one per column of the operand, and each block generally sits at its own global columns, so no single triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$ describes it. Merging the blocks into one larger triple would store the full block diagonal, p copies of $\mathbf{A}$ separated by blocks of zeros, and falling back to generic sparse would lose the dense kernels. We instead keep the blocks as they are, in a third representation.

A *stacked permuted dense* matrix is a vertical union of permuted dense blocks whose row vectors are pairwise disjoint; the column vectors may overlap. The blocks share one value buffer. Every kernel treats the blocks one at a time. Structural atoms rewrite the index vectors of each block, and products distribute over the blocks, with one `dgemm` for each pair of blocks with intersecting row sets and nothing at all for the pairs with disjoint row sets, as in (3.2). The assembled matrix is never allocated or traversed, and each stacked kernel is a short loop over the corresponding permuted dense kernel. Figure 3.5 shows the Jacobian of $\mathbf{AX}$ for a variable $\mathbf{X}$ with two columns: one block per column of $\mathbf{X}$, each a copy of $\mathbf{A}$ at its own rows and columns.

$$\mathbf{M} = \begin{pmatrix} a_{00} & a_{01} & \cdot & \cdot & \cdot & \cdot \\ a_{10} & a_{11} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & a_{00} & a_{01} & \cdot & \cdot \\ \cdot & \cdot & a_{10} & a_{11} & \cdot & \cdot \end{pmatrix}$$

$$\mathbf{X}_1 = \mathbf{X}_2 = \mathbf{A}, \qquad \mathbf{r}_1 = (0,1), \quad \mathbf{c}_1 = (0,1), \qquad \mathbf{r}_2 = (2,3), \quad \mathbf{c}_2 = (2,3).$$

Figure 3.5 The Jacobian of $\mathbf{AX}$ as a stacked permuted dense matrix, for dense $\mathbf{A} \in \mathbb{R}^{2\times 2}$ and a variable $\mathbf{X} \in \mathbb{R}^{2\times 2}$ in a problem with six variables. There is one block per column of $\mathbf{X}$, each a permuted dense copy of $\mathbf{A}$ at its own rows and columns. The 4×6 matrix $\mathbf{M}$ is never assembled; a product with $\mathbf{M}$ is one `dgemm` per block.

Generic sparse, permuted dense, and stacked permuted dense are the only three matrix representations in the engine. Generic sparse is the default, produced by the variable leaves and by every atom that does not hold dense data. The other two are created by `left_matmul` and the quadratic form, the two atoms that hold dense data, and propagate from there through the structural atoms and the chain rule.

3.3 Computational kernels

A *kernel* is an operation on one or two of the matrix representations of §3.2. There is one kernel per operation and representation rather than one per atom. Transposition, row indexing, broadcasting, row partition sums, and the two products of §3.2.1 cover every atom in the library. Each kernel is a pair of functions, one called by the symbolic pass, which determines the structure of the result and reserves its memory, and one called by the numeric pass, which writes values into that fixed structure.

3.3.1 Kernel dispatch

Unary operations are dispatched on the representation of their single operand, and so is $\mathbf{BA}$, whose left operand is always the data block of an atom. The transposed products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{DA}$ depend on the representations of both operands, which a single table cannot express. They are instead resolved with double dispatch. A dispatch function switches on the type tags of $\mathbf{B}$ and then of $\mathbf{A}$, and calls one of nine kernels. Figure 3.6 shows the representation of the result for the nine combinations of operand types.

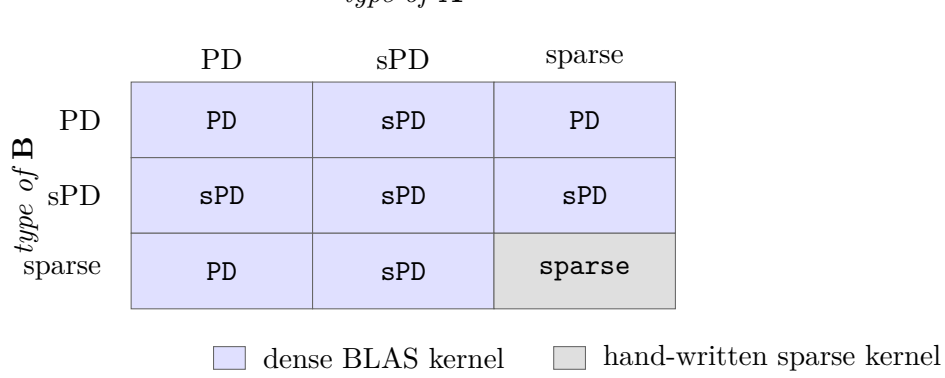


Figure 3.6 Double dispatch of the transposed products $\mathbf{B}^\top\mathbf{A}$ and $\mathbf{B}^\top\mathbf{DA}$. Each cell names the representation of the result for one combination of operand types (PD is permuted dense, sPD is stacked permuted dense); its colour indicates the kernel family that computes it. Only the sparse-sparse combination falls back to a hand-written sparse kernel.

A product with a permuted dense or stacked permuted dense operand produces a result of the same kind, so once a dense block has been identified, it remains in a structured representation through all later products. Eight of the nine kernels are based on dense BLAS calls; in the mixed combinations, the columns of the sparse operand are gathered into the dense computation. Only the product of two generic sparse matrices uses Gustavson's algorithm [55], with the symbolic and numeric phases discussed in §1.4.3.

3.3.2 Reusing work across passes

The Jacobian of an affine atom does not depend on `x`. For the structural atoms, such as indexing, transposition, reshaping, stacking, and summation, it is determined by shapes and index sets alone. The remaining affine atoms, such as `left_matmul`, `scalar_mult`, `convolve`, and `kron`, place problem data in the Jacobian, whose values only change when a parameter update sets their refresh flag.

The numeric pass never allocates memory; every buffer, including the scratch space of the kernels, is allocated during the symbolic pass. Beyond allocation, several other kinds of work are done only once. Sparsity patterns are computed symbolically: `add` merges the patterns of its children with a sorted merge, the sparse products run the symbolic phase of Gustavson's algorithm [58], and the factored products of §3.2 determine their output index vectors before any values exist. Index maps are also precomputed. Transposition stores its row permutation, `kron` its per-row copy maps, and `sum` the map from child nonzeros to output entries, so the numeric pass performs no index arithmetic. When the index vectors of two operands do not intersect, the product is recorded as empty during the symbolic pass, and its numeric pass does nothing. Finally, values that cannot change are written once; the Jacobian of a variable, for example, is filled during the symbolic pass and never touched again.

Figure 3.7 illustrates this for the `sum` atom. During the symbolic pass, the representation of the child allocates the output and fills `idx_map`, an array that maps each nonzero of the child Jacobian to the output entry it is added to. The axis convention and all other index decisions are handled at this point. The numeric pass is a single loop, the same for every axis and every representation. Indexing, transposition, and `kron` work the same way, with a copy instead of an addition.

3.3.3 Problem assembly and parameter updates

The same split between structure and values is applied at the problem level. The solver receives one global CSR matrix, formed by concatenating the constraint Jacobians, whose layout is fixed during the symbolic pass; an evaluation only copies the values of each constraint root into this matrix at a fixed offset. Affine constraints are skipped entirely after the first evaluation, because their values cannot change until the parameters do.

A parameter update only involves the numeric pass. The function `update_params` writes the new values into the parameter leaves and sets a refresh flag on each affected subtree, and the next evaluation recomputes the values through the fixed maps and kernels. On the `ignore_dpp` path of §2.1.3, `CVXPY` instead repeats the whole matrix stuffing pass, recomputing

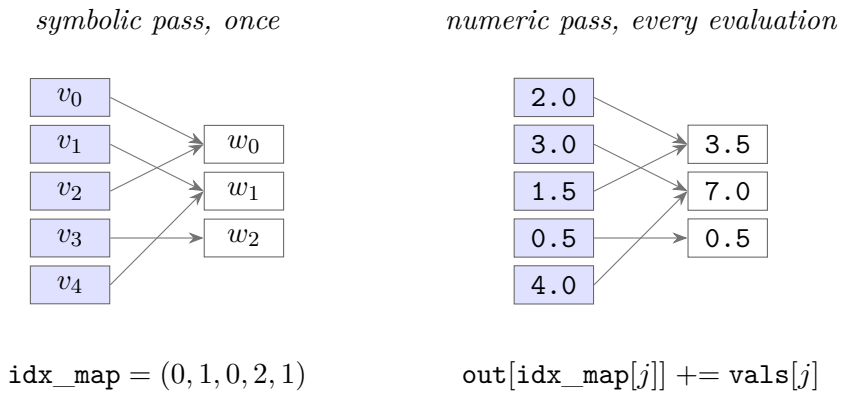


Figure 3.7 The two passes of the `sum` atom. During the symbolic pass, `idx_map` is built, shown as arrows from the nonzeros v_j of the child Jacobian to the output entries w_i; at every evaluation, the values are accumulated along the fixed arrows. The numeric passes of the other structural atoms have the same form.

every sparsity pattern and reallocating every buffer, each time a parameter changes. The difference is measured in Chapter 4.

3.4 From `CVXPY` to the engine

The engine is a C library, and `sparsediffpy` is the Python extension module that exposes it. The module is written against the C interfaces of `Python` and `NumPy`. It declares one function per atom constructor, together with the problem-level entry points, and each of these functions reads its arguments, calls the corresponding C function, and returns an opaque handle to the node it created. Three thousand lines of binding code cover the seventeen thousand lines of the engine, and none of them mention `CVXPY`.

Everything specific to `CVXPY` is on the other side, in a converter that turns a canonicalized problem into an engine graph. The converter is used on two paths. The nonlinear programming interface [34] builds a graph from the smooth reformulation of a problem. The canonicalization backend, named `diffengine` and measured in Chapter 4, builds a graph from the conic form and extracts the problem data from it, in place of the matrix stuffing pass of §2.1. It is selected by passing `ignore_dpp=True` to `CVXPY`. The two paths build the graph in the same way, and differ in what they ask of it.

The conversion is a single pass over the tree. At a node, the children are converted first, and the name of the atom then selects the engine constructor to call on the resulting handles. That dispatch is a table with about fifty entries, one for each atom of `CVXPY` that the engine implements. The following listing is condensed from the converter.

```
def convert(expr, var_dict, param_dict, n_vars):
    if isinstance(expr, cp.Variable): # leaves are looked up,
        return var_dict[expr.id]      # never rebuilt
    if isinstance(expr, cp.Parameter):
        return param_dict[expr.id]
    if isinstance(expr, cp.Constant): # a constant is a parameter
        d1, d2 = shape_2d(expr.shape)  # with the reserved offset -1
        return engine.make_parameter(d1, d2, -1, n_vars, dense(expr.value))

    children = [convert(arg, var_dict, param_dict, n_vars)
                for arg in expr.args]
    return ATOM_CONVERTERS[type(expr).__name__](expr, children)

ATOM_CONVERTERS = {
    "AddExpression": lambda expr, children: chain_add(children),
    "MulExpression": convert_matmul,
    "index":        convert_index,
    "QuadForm":     convert_quad_form,
    "exp":          lambda expr, children: engine.make_exp(children[0]),
}
```

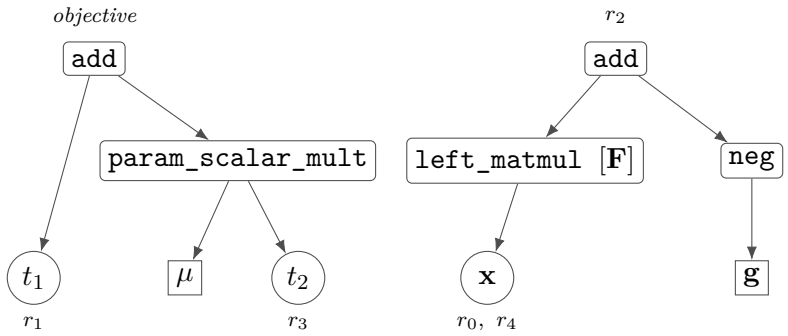
Two checks are left out of the listing. Every constructor reports the dimensions of the node it created, which are compared against the shape that `CVXPY` assigned to the expression. A subtree that contains neither a variable nor a parameter is evaluated once and folded into a single constant leaf, because canonicalization leaves such subtrees in place.

The two leaf dictionaries are built before the pass, from the inverse data that `CVXPY` produces when it stacks the variables of the problem. A variable leaf is created with its column offset in the stacked variable vector, the offsets of Figure 2.2, and the engine uses that offset directly as the first column of the selection Jacobian of §3.1.2. A parameter leaf is created with its offset in the flattened parameter vector $\boldsymbol{\theta}$, which is where `update_params` reads its new values. Because the dictionaries are built once and looked up by identity, every occurrence of a variable converts to the same node, and the trees of the objective and of the constraints become the single graph of §3.1.2.

The conversion is also where the representations of §3.2 are chosen. A multiplication by problem data is converted by looking at which operand is constant and at how that operand is stored. Sparse data is passed as a CSR triple, dense data as a dense block, and a dense array whose entries are mostly zeros is passed as a CSR triple as well, after its nonzeros have

been counted. The dense block is what the atom later places in a permuted dense Jacobian. A sum of several terms is converted to a balanced tree of binary additions, so the depth of the graph grows with the logarithm of the number of terms.

The objective converts to one root, and each constraint contributes one root per argument, in the order in which `CVXPY` groups the cones: zero, nonnegative, second-order, semidefinite, exponential, and power. That order fixes the row blocks of the constraint Jacobian assembled in §3.3.3. The engine has no notion of a cone. It receives an objective root and an array of constraint roots, and the cone that each block of rows belongs to is recorded on the `CVXPY` side. Figure 3.8 shows the graph and the roots of the running example.



	Root	Rows	Cone
objective	$t_1 + \mu t_2$		
r_0	$\mathbf{x}$	$0, \ldots, n-1$	$\mathbb{R}^n_+$
r_1	t_1	n	$\mathcal{K}_{\text{soc},m+1}$
r_2	$\mathbf{Fx} - \mathbf{g}$	$n+1, \ldots, n+m$	$\mathcal{K}_{\text{soc},m+1}$
r_3	t_2	$n+m+1$	$\mathcal{K}_{\text{soc},n+1}$
r_4	$\mathbf{x}$	$n+m+2, \ldots, 2n+m+1$	$\mathcal{K}_{\text{soc},n+1}$

Figure 3.8 The conic form of the running example (1.5), whose expression trees are in Figure 1.3, after conversion. The atoms are the engine nodes, the circles are variables and the boxes are parameters; `left_matmul` holds the data matrix $\mathbf{F}$ as a dense block. The variable $\mathbf{x}$ is converted once and reached from three places, so the four trees form one graph. The table lists the roots in the order the constraints are grouped, which is the order of the row blocks of $[\mathbf{A}\ \mathbf{b}]$ in Figure 2.3.

A solver reaches the graph through an oracle interface.

```
rows, cols = oracles.jacobianstructure() # once, fixed afterwards
f    = oracles.objective(x)   # at each iterate
h    = oracles.constraints(x)
grad = oracles.gradient(x)    # reuses the cached forward values
vals = oracles.jacobian(x)    # aligned with (rows, cols)
```

The pattern is determined when the graph is built, and every later call returns a value array aligned with it. Within an iterate, the forward values are computed by the first two calls and reused by the other two, so the gradient and the Jacobian perform no forward pass of their own. The Hessian of the Lagrangian is queried in the same way, with one call for its pattern and one for its values.

The canonicalization backend asks the same questions exactly once, and at the origin. Write f_0 for the converted objective and $\mathbf{h}$ for the stack of converted constraint roots. After canonicalization $\mathbf{h}$ is affine and f_0 is affine or quadratic, so

$$d = f_0(\mathbf{0}), \qquad \mathbf{c} = \nabla f_0(\mathbf{0}), \qquad \mathbf{P} = \nabla^2 f_0, \qquad \mathbf{b} = \mathbf{h}(\mathbf{0}), \qquad \mathbf{A} = \nabla \mathbf{h}(\mathbf{0}), \tag{3.4}$$

which is the problem data of (1.2). These are the matrices that Figure 2.3 assembles by matrix stuffing, obtained instead as the derivatives of §1.3, and (3.4) is the identity that the `CasADi` listing of §2.2.3 writes with `jacobian` and `substitute`. The sign convention of the cone form and the reordering of the rows inside each cone are applied afterwards, by the step that formats the constraints for the other backends as well. The sparsity patterns of $\mathbf{A}$ and $\mathbf{P}$ are read once, when the graph is built, and every later evaluation returns value arrays aligned with them.

New parameter values go through one method.

```
def apply_parameters(self, id_to_value):
    theta = flatten_and_concatenate(id_to_value)
    if np.array_equal(theta, self.extracted_theta): # unchanged values:
        return self.P, self.c, self.d, self.A, self.b # reuse everything
    self.extractor.update_parameters(theta)     # numeric pass only
    self.c, self.d, self.A, self.b, self.P = self.extractor.extract()
    self.extracted_theta = theta
    return self.P, self.c, self.d, self.A, self.b
```

Repeated solves at unchanged parameter values return the cached matrices. An actual update costs one numeric pass over the graph and one evaluation of (3.4), while the graph, its sparsity patterns and its buffers are built once and live as long as the problem family. Chapter 4 measures what this is worth against `CVXPY`'s canonicalization backends, implementations of the matrix-stuffing approach reviewed in Chapter 2.

CHAPTER 4 BENCHMARK RESULTS

This chapter presents benchmark results which compare the `diffengine` backend (the `CVXPY` implementation based on `SparseDiffEngine`) against `CVXPY`'s own canonicalization backends: the default `cvxcore` backend, the matrix-stuffing baseline of §2.1, and the two pure-Python backends, `SCIPY` and `COO`, that `CVXPY` ships as alternatives to it. The comparison is restricted to `CVXPY`'s own pipeline because that is the claim under test: `diffengine` is a drop-in replacement for the canonicalization backend, and the relevant question is whether it constructs the same problem data faster. All backends run on the 25 problems of `CVXPY`'s public benchmark suite (`https://github.com/cvxpy/benchmarks`). These convex optimization problems cover many fields, ranging from portfolio models, robust regression, and semidefinite programs to simple LPs and QPs that are close to standard form.

The suite divides naturally into two groups, and this chapter treats them separately because they pose different questions. On the 17 problems whose data are constants, there is exactly one question: what does a single cold construction of (A, b, c, P) cost? On the 8 problems carrying `cp.Parameter` objects there are two, because `CVXPY` can either build the problem data directly or build a *parameter-to-data tensor* that is reused when parameter values change. The first question is then how the backends compare when all of them construct the same object, and the second is when the extra cost of building that tensor repays itself. §4.1 describes the measurement protocol, §4.2 covers the non-parametric problems, and §4.3 the parametric ones.

4.1 Benchmark Setup

Every benchmark measures the construction of the problem data (A, b, c, P); no solver is invoked. For all backends this corresponds to a call to `get_problem_data` with the target solver fixed to `Clarabel` [23], and that single call is the entire timed region. The `SCIPY` and `COO` backends are selected through the `canon_backend` argument of the same call, and `diffengine` by passing `ignore_dpp=True`.

4.1.1 Comparing like with like

A backend on the default path does not necessarily produce the same object as `diffengine`. When a problem carries parameters, `CVXPY`'s *disciplined parametrized programming* (DPP) path builds a parameter-to-data tensor, from which the problem data for a given parameter

value is recovered by a tensor–vector product. `diffengine` instead keeps the expression tree and emits the concrete matrices directly. Timing one against the other measures two different workloads, and flatters whichever is doing less.

To compare like with like, every parametric measurement in §4.3 is also taken with `ignore_dpp=True` on the *stock* backends, written `CPP_ND`, `SCIPY_ND` and `COO_ND`. On that path `CVXPY` substitutes each parameter by its value before canonicalizing, so the backend builds a plain, non-parametric matrix — the same object `diffengine` produces. That the two are genuinely the same object is verified rather than assumed: for ConvexPlasticity, ParamConeMatrixStuffing and ParamSmallMatrixStuffing, the A matrices agree bit-identically ($\max|\Delta| = 0$) and P to 5.6×10^{-17}.

The `_ND` columns require stock `CVXPY`, because the `diffengine` development fork rejects an explicit backend on the `ignore_dpp` path. They are therefore measured in a second environment, whose offset against the first is quantified below.

4.1.2 Two baselines for `cvxcore`

`CVXPY` does not always use the backend it is asked for. When no `canon_backend` is given and the total parameter size reaches 1000 entries, it silently substitutes `COO` for `cvxcore`. On the parametric problems the two therefore differ by more than an order of magnitude, and a column labelled `CPP` that was obtained without pinning the backend is not measuring `cvxcore` at all. Both are reported below: `default`, which is what a user gets and hence the right baseline for a drop-in-replacement claim, and `CPP`, obtained by passing `canon_backend="CPP"` explicitly.

4.1.3 Protocol and environment

Each cold compilation is measured once, in a fresh process per backend so that no cached compilation chain is reused; the re-solve columns are means of three re-evaluations. All measurements run single-threaded (BLAS thread pools pinned to one thread) on a 2018 MacBook Pro with a 6-core Intel Core i7-8850H at 2.6 GHz and 16 GB of RAM, under macOS 15.7. Two environments are used:

- *engine*: the `diffengine` development fork of `CVXPY`, `sparsediffpy` 0.7.0, `NumPy` 2.4 [39] and `SciPy` 1.17 [57];
- *stock*: `CVXPY` 1.9.2 as released, `NumPy` 2.5, `SciPy` 1.18, used for the `_ND` columns.

Both run `Python` 3.14.

Two properties of the measurements bound what can be claimed from them. The first is measurement noise. On the non-parametric problems `ignore_dpp` is a documented no-op, so `X` and `X_ND` must agree; they do, with a geometric mean of 0.99, 1.02 and 1.01 for the three backends, though individual pairs range over 0.74–1.18 because the two halves of each pair are measured far apart in a sweep that runs for hours. Replicating the parametric measurements three times gives a sharper picture: a single cell repeats to within 1–8 %, a ratio formed inside one environment to within 1–4 %, and a ratio spanning the two environments to within 3–19 %. Ratios are quoted below at that resolution and no finer.

The second is a systematic offset: for identical backends on identical problems the engine environment runs 1.03–1.21× slower than the stock one. Comparisons that cross the two are therefore conservative, the offset working against `diffengine`.

The scripts which run the benchmarks and all recorded timings are available at `https://github.com/SparseDifferentiation/Jacobian-accumulation` under the `canonicalization/` directory; every ratio quoted in this chapter is reproducible from the committed result files.

4.2 Non-parametric problems

On the 17 problems whose data are constants no parameter tensor exists, so all four systems construct the same object and the comparison needs no adjustment (Table 4.1). The ratio is taken against the *fastest* of `CVXPY`'s three backends on each problem rather than against `cvxcore`, because `cvxcore` is frequently not the fastest of them.

`diffengine` is the fastest of the four on 14 of the 17 problems, with a geometric-mean ratio of 0.32, that is, 3.1× faster than an oracle that picks the best `CVXPY` backend for each problem. The largest margins occur where the baselines' per-expression tensor construction dominates, as in the portfolio and regression models with many scalar constraints; UnconstrainedQP, a tiny problem wrapped in a large `kron` expression, is the extreme case at 121× against the best backend (146× against `cvxcore`).

The three exceptions share a mechanism. CVaR (1.01), OptimalAdvertising (1.05) and SDPSegfault1132 (1.39) are the problems whose compilation is dominated by moving one large block of constant data into the problem matrices rather than by computing coefficients. Where there are no coefficients to derive, structural knowledge of the atoms buys nothing, and the comparison reduces to how efficiently each system copies memory.

The table also shows that `cvxcore` is not uniformly the fastest of `CVXPY`'s own backends: the pure-Python `SCIPY` backend beats it on 14 of the 17 problems, usually modestly but by 6.8×

Table 4.1 Cold compilation time on the 17 non-parametric problems. The ratio is `diffengine` over the best of `CVXPY`'s three backends on each problem; values below 1 mean `diffengine` beats all of them. The final row is the geometric mean of the per-problem ratios.

Benchmark	`CPP` (s)	`SCIPY` (s)	`COO` (s)	`diffengine` (s)	Ratio
UnconstrainedQP	10.32	8.52	13.54	0.07	**0.01**
LeastSquares	9.61	10.08	10.68	1.45	0.15
Yitzhaki	3.56	3.28	3.55	0.51	0.16
HuberRegression	12.05	10.72	11.60	1.73	0.16
SimpleQP	11.27	10.59	11.97	1.87	0.18
SimpleLP	37.57	29.99	30.96	6.12	0.20
TvInpainting	3.89	3.50	3.57	1.20	0.34
Cajas	22.67	2.91	2.89	1.07	0.37
SlowPruning	7.69	7.82	8.60	3.01	0.39
SmallMatrixStuffing	6.97	7.09	10.84	2.76	0.40
QuantumHilbertMatrix	4.27	4.05	4.75	1.67	0.41
ConeMatrixStuffing	11.06	9.34	16.99	3.95	0.42
SemidefiniteProgramming	5.96	2.43	3.78	2.02	0.83
Murray	7.13	3.87	4.00	3.72	0.96
CVaR	35.85	24.79	26.86	24.94	1.01
OptimalAdvertising	6.43	2.32	2.25	2.36	1.05
SDPSegfault1132	64.84	9.51	43.00	13.21	**1.39**
Geometric mean					**0.32**

on SDPSegfault1132 and 7.8× on Cajas. This is why the ratio above is taken against the best of the three. Had `cvxcore` alone been the baseline the geometric mean would have been 0.20 rather than 0.32 — a difference attributable entirely to the choice of baseline, not to `diffengine`.

4.3 Parametric problems

The 8 parametric problems admit the two questions raised at the start of this chapter. §4.3.1 isolates what the parameter tensor costs to build, §4.3.2 compares the backends once that difference is removed, and §4.3.3 asks when the tensor repays itself.

4.3.1 The cost of building the parameter tensor

Table 4.2 reports, for each stock backend, the ratio of its DPP time to its `ignore_dpp` time on the same problem in the same environment. The ratio isolates the tensor: numerator and denominator differ only in whether one is built.

Table 4.2 Cost of building the parameter tensor. Each entry is the backend's DPP time divided by its `ignore_dpp` time on the same problem; values above 1 mean the tensor is more expensive than constructing the data directly. Measured in the stock environment.

Benchmark	#par. entries	`CPP`	`SCIPY`	`COO`
ParamSmallMatrixStuffing	90 450	**27.7**	**60.2**	0.82
ParamConeMatrixStuffing	40 300	**17.8**	**23.7**	0.80
ParametrizedQP	14 406 000	—	—	0.99
SimpleFullyParametrizedLP	1 000 000	—	—	0.98
SimpleScalarParametrizedLP	1	1.04	1.01	1.11
FactorCovarianceModel	2	1.01	1.03	0.99
SVMWithL1Regularization	1	0.97	0.92	0.96
ConvexPlasticity	—	1.01	1.01	1.01
Geometric mean		**2.82**	**3.34**	**0.95**

The tensor is not intrinsically expensive: it is expensive *for two of the three backends*. `CPP` and `SCIPY` pay 18–60× on the two problems with tens of thousands of parameter entries, while `COO` pays nothing anywhere — on those same problems building the tensor is marginally *cheaper* than building the data directly. This is precisely what `COO`'s coordinate-format tensor is designed for, its operations scaling with the number of stored nonzeros rather than with the parameter dimension (§1.4.2). On the two problems with the largest parameter counts the degradation is not merely a slowdown but an outright failure, which is why those four

cells are blank: pinned to `cvxcore`, SimpleFullyParametrizedLP terminates the worker process, on both the stock and the development environment, and an attempt to measure ParametrizedQP the same way ended with the entire benchmark run being killed. `SCIPY` fails on both as well — it crashes the worker on the first and was still running on the second after 77 min — reproducing the same two failures recorded in earlier sweeps. `COO` completes both without difficulty.

This explains the substitution rule of §4.1.2. `CVXPY` switches to `COO` above 1000 parameter entries because that is exactly the regime in which the other two backends degrade. A consequence worth stating explicitly is that the default path never pays the tensor penalty: it routes around it. ConvexPlasticity is a further special case — it is not DPP under the criterion the solving chain applies, so parameters are substituted regardless of the flag and all three columns coincide.

4.3.2 Cold construction, compared like with like

Table 4.3 compares `diffengine` against the stock backends on the `ignore_dpp` path, where all four construct the same matrices.

Table 4.3 Cold construction on the parametric problems, all systems producing the same non-parametric matrices. `default` is `CVXPY` with no backend specified. The ratio is `diffengine` over the best of the three `_ND` columns. Entries are medians of three replications except where marked; a dash marks a measurement that did not complete, see the text.

Benchmark	`default`	`CPP_ND`	`SCIPY_ND`	`COO_ND`	`diffengine`	Ratio
FactorCovarianceModel	5.43	5.32	4.32	4.67	0.87	**0.20**
SVMWithL1Regularization	11.52	11.21	8.31	9.79	1.87	0.22
SimpleScalarParametrizedLP	4.74	4.56	4.12	4.25	1.03	0.25
SimpleFullyParametrizedLP†	2.16	—	2.16	2.22	0.54	0.25
ParametrizedQP†	10.28	—	9.68	10.76	2.53	0.26
ParamSmallMatrixStuffing	0.43	0.28	0.50	0.49	0.21	0.76
ParamConeMatrixStuffing	0.29	0.18	0.32	0.32	0.14	0.79
ConvexPlasticity	2.09	0.35	0.35	0.35	2.07	**5.97**
Geometric mean						**0.48**

† Single measurement; all other rows are medians of three replications.

`diffengine` is fastest on seven of the eight problems, with a geometric mean of 0.48, that is, 2.1× faster than the best stock backend. The margin is smaller than on the non-parametric problems (0.32), and the reason is visible in the table: the two matrix-stuffing problems are small enough (0.14–0.21 s) that fixed overhead in `CVXPY`'s Python-side reduction chain, which

every column pays, compresses the achievable ratio. Excluding those two, the geometric mean is 0.40.

Replicating each of these measurements three times also settles how far the ratios can be trusted. Repeat timings of one cell vary by 1–8 %, and ratios formed within a single environment are tighter still — the `CPP_ND/CPP` entries of Table 4.2 reproduce to 1–4 % on five of six problems, because numerator and denominator are measured moments apart and any machine drift cancels. The ratios in this table are the loosest, spanning 3–19 % across replications, since their two operands come from different environments and nothing cancels. That is the resolution at which the table should be read: the ordering is reliable, and so is the distinction between 0.2 and 0.8, but 0.76 against 0.79 is not a meaningful gap.

The two dashes are the `CPP_ND` cells of the problems discussed in §4.3.1, where pinned `cvxcore` does not complete. Both rows are nonetheless included, because the remaining two `_ND` columns still bound the comparison: whatever `cvxcore` would have taken, it could only have been slower than the 2.16 s and 9.68 s used as baselines here, so the ratios shown are upper bounds on `diffengine`'s advantage. Note also that these two problems, which carry by far the largest parameter counts in the suite, give `diffengine` its second and third best ratios — had they been omitted for want of a complete row, the reported geometric mean would have been worse, not better.

ConvexPlasticity is the one problem in the entire suite where `diffengine` is clearly beaten, and by a wide margin: 2.07 s against 0.35 s, a factor of 6.0 reproduced to within 4 % across replications. Earlier measurements did not show this, because on the development fork the unpinned column is itself routed to `diffengine` for this problem, so the baseline and the system under test were the same code. The cause is not derivative computation. `diffengine` emits a P matrix with 9 048 000 stored entries where only 33 000 are nonzero — a 274× padding — which `SciPy` must then re-canonicalize; the stock backends emit exactly the 33 000. It is a materialization defect with a known remedy, not a limit of the approach, and it is discussed further in §5.2.

4.3.3 When the parameter tensor repays itself

The tensor exists to be reused. Its cost is paid once and recovered over subsequent re-solves, so the honest question is not which construction is faster but after how many re-solves the investment breaks even. Table 4.4 answers it: the break-even count is the extra cost of building the tensor divided by the saving it yields per re-solve.

Two regimes appear. Where the parameters are a handful of scalars, the tensor is essentially

Table 4.4 Amortization of the parameter tensor. "Extra cold" is the DPP construction time minus the direct construction time; "saved" is the per-re-solve advantage of applying a cached tensor over reconstructing from scratch. "Break-even" is their quotient, in re-solves. The last column gives the same count for `diffengine` against the tensor.

Benchmark	Extra cold (s)	Saved / re-solve (s)	Break-even	vs `diffengine`
ParamConeMatrixStuffing	3.07	0.162	19 re-solves	never
ParamSmallMatrixStuffing	7.61	0.234	33 re-solves	never
SimpleScalarParametrizedLP	0.63	4.28	immediate	never
FactorCovarianceModel	0.97	4.20	immediate	25 re-solves
SVMWithL1Regularization	3.80	8.66	immediate	32 re-solves

free to build and saves seconds on every re-solve, so it pays for itself immediately; a workflow that re-solves even once should use it. Where the parameters are large, building the tensor costs seconds and saves only fractions of one, and 19 to 33 re-solves are needed before it is worthwhile.

Against `diffengine` the comparison is more favourable still. On three of the five problems `diffengine` is faster both cold and warm, so no number of re-solves lets the tensor overtake it; on the remaining two it would take 25 and 32 re-solves. The engine occupies a middle position by construction: it never builds a tensor, so it never pays for one, but it caches the compiled program and re-evaluates only the parameter-dependent coefficients, which recovers most of what the tensor buys. Since these counts compare across the two environments of §4.1, and the engine environment is the slower of the two, they understate `diffengine`'s position.

4.4 Summary

Measured against an oracle that selects the best of `CVXPY`'s three canonicalization backends for each problem, `diffengine` constructs the problem data 3.1× faster on the non-parametric problems and 2.1× faster on the parametric ones, and it is the fastest system on 21 of the 25 problems in the suite. Both figures are conservative in the same direction: the baseline is an oracle no user actually has, and the engine's timings come from the slower of the two measurement environments.

The exceptions are as informative as the wins, and they are consistent with the design described in Chapter 3. `diffengine` wins where problem data must be *derived*, because it reads each atom's Jacobian block off structurally instead of reconstructing it; it draws level where the work is *copying* a large constant block (CVaR, OptimalAdvertising,

SDPSegfault1132), where no structural knowledge helps; and it loses once, on Convex-Plasticity, to a defect in how a sparse matrix is materialized rather than to anything about computing derivatives.

Two limitations bound these results. The parametric measurements are medians of three replications and repeat to within 3–19 %, but those of Table 4.1 are single draws; the ratios near unity there distinguish "comparable" from "clearly faster" but should not be read more finely than that. And several cells are missing, in every case because a stock backend failed to complete rather than because the measurement was not attempted: `cvxcore` and `SCIPY` both fail on SimpleFullyParametrizedLP and ParametrizedQP, the two problems with the largest parameter counts. Each omission removes a slow baseline, so every affected ratio understates `diffengine`'s advantage.

Peak memory is measured by the same harness and is not reported here: it does not follow the timing results — on several problems `diffengine` is markedly heavier — and it is taken up separately in §5.2.

CHAPTER 5 CONCLUSION

5.1 Summary of Works

5.2 Limitations

5.3 Future Research

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," Alfred P. Sloan School of Management, Massachusetts Institute of Technology, Cambridge, MA, Tech. Rep., 1978.

[2] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[3] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[4] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[5] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[6] J. Bisschop, *AIMMS Optimization Modeling*. Haarlem, The Netherlands: Paragon Decision Technology, 2006.

[7] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[8] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[9] M. L. Bynum *et al.*, *Pyomo—Optimization Modeling in Python*. Springer, 2021.

[10] W. E. Hart *et al.*, "Pyomo: modeling and solving mathematical programs in python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[13] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[14] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[15] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[16] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[17] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[18] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[19] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[20] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[21] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[22] D. Ge, Q. Huangfu, Z. Wang, J. Wu, and Y. Ye, "Cardinal Optimizer (COPT) user guide," *arXiv preprint arXiv:2208.14314*, 2022.

[23] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.

[24] A. Ben-Tal and A. Nemirovski, *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*. SIAM, 2001.

[25] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[26] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[27] W. Zhang, P. Nobel, A. Jeendgar, R. Murray, P. Schiele, and S. Diamond, "CVXPY 1.9: Recent advances in optimization modeling software," 2026. [Online]. Available: https://arxiv.org/abs/2606.14891

[28] A. R. Curtis, M. J. D. Powell, and J. K. Reid, "On the estimation of sparse Jacobian matrices," *IMA Journal of Applied Mathematics*, vol. 13, no. 1, pp. 117–119, 1974. [Online]. Available: https://doi.org/10.1093/imamat/13.1.117

[29] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[30] D. M. Gay, "Writing .nl files," Sandia National Laboratories, Tech. Rep. SAND2005-7907P, 2005, describes AMPL's expression DAG and coefficient extraction internals.

[31] D. M. Gay, "Hooking your solver to AMPL," Bell Laboratories, Tech. Rep. 97-4-06, 1997, reference for the AMPL Solver Library (ASL).

[32] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[33] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

[34] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2025. [Online]. Available: https://arxiv.org/abs/2606.02896

[35] E. M. B. Smith, "On the optimal design of continuous processes," Ph.D. dissertation, Imperial College London (University of London), London, 1996, ph.D. dissertation.

[36] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[37] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[38] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[39] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[40] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[41] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[42] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[43] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[44] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario, 2026. [Online]. Available: https://www.maplesoft.com

[45] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[46] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[47] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[48] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for Fortran usage," *ACM Transactions on Mathematical Software*, vol. 5, no. 3, pp. 308–323, 1979.

[49] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Transactions on Mathematical Software*, vol. 16, no. 1, pp. 1–17, 1990.

[50] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide*, 3rd ed. Philadelphia, PA: SIAM, 1999.

[51] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[52] H. M. Markowitz, "The elimination form of the inverse and its application to linear programming," *Management Science*, vol. 3, no. 3, pp. 255–269, 1957.

[53] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[54] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[55] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software*, vol. 4, no. 3, pp. 250–269, 1978.

[56] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[57] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[58] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Advances in Computational Mathematics*, vol. 1, no. 1, pp. 127–137, 1993.

[59] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," 2010. [Online]. Available: http://eigen.tuxfamily.org

[60] I. S. Duff, M. A. Heroux, and R. Pozo, "An overview of the sparse basic linear algebra subprograms: The new standard from the BLAS technical forum," *ACM Transactions on Mathematical Software*, vol. 28, no. 2, pp. 239–267, 2002.

[61] J. R. Magnus and H. Neudecker, *Matrix Differential Calculus with Applications in Statistics and Econometrics*, 3rd ed., ser. Wiley Series in Probability and Statistics. John Wiley & Sons, 2019.

[62] P. S. Dwyer and M. S. Macphail, "Symbolic matrix derivatives," *The Annals of Mathematical Statistics*, vol. 19, no. 4, pp. 517–534, 1948.

[63] The CVXPY developers, "CVXPY source code repository," https://github.com/cvxpy/cvxpy, 2026.

[64] The CVXPY developers, "CVXPY documentation," https://www.cvxpy.org/, 2026.

[65] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[66] A. Hill and G. Dalle, "Sparser, better, faster, stronger: Sparsity detection for efficient automatic differentiation," *arXiv preprint arXiv:2501.17737*, 2025. [Online]. Available: https://arxiv.org/abs/2501.17737

[67] M. Blondel and V. Roulet, "The elements of differentiable programming," 2024, arXiv:2403.14606. [Online]. Available: https://arxiv.org/abs/2403.14606

[68] A. Griewank and S. Reese, "On the calculation of Jacobian matrices by the Markowitz rule," in *Automatic Differentiation of Algorithms: Theory, Implementation, and Application*, A. Griewank and G. F. Corliss, Eds. Philadelphia, PA: SIAM, 1991, pp. 126–135.

[69] U. Naumann, "Optimal Jacobian accumulation is NP-complete," *Mathematical Programming*, vol. 112, no. 2, pp. 427–441, 2008. [Online]. Available: https://doi.org/10.1007/s10107-006-0042-z

[70] A. Griewank, D. Juedes, and J. Utke, "Algorithm 755: ADOL-C: a package for the automatic differentiation of algorithms written in C/C++," *ACM Transactions on Mathematical Software*, vol. 22, no. 2, pp. 131–167, 1996. [Online]. Available: https://doi.org/10.1145/229473.229474

[71] B. M. Bell, "CppAD: A package for C++ algorithmic differentiation," 2023, computational Infrastructure for Operations Research. [Online]. Available: https://github.com/coin-or/CppAD

[72] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[73] T. F. Coleman and J. J. Moré, "Estimation of sparse Jacobian matrices and graph coloring problems," *SIAM Journal on Numerical Analysis*, vol. 20, no. 1, pp. 187–209, 1983. [Online]. Available: https://doi.org/10.1137/0720013

[74] M. J. D. Powell and P. L. Toint, "On the estimation of sparse Hessian matrices," *SIAM Journal on Numerical Analysis*, vol. 16, no. 6, pp. 1060–1074, 1979. [Online]. Available: https://doi.org/10.1137/0716078

[75] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J. on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[76] CasADi developers, "CasADi user guide," https://web.casadi.org/docs/, 2025.

[77] AMPL Optimization Inc., "amplpy: Python API for AMPL," https://amplpy.ampl.com, 2025, python interface to the AMPL modeling language.

[78] AMPL Optimization Inc., "MP: modeling guide for MP-based AMPL solvers," https://mp.ampl.com/model-guide.html, 2026.

[79] D. M. Gay, "Automatic differentiation of nonlinear AMPL models," AT&T Bell Laboratories, Tech. Rep. Numerical Analysis Manuscript 91-05, 1991, reverse mode AD on AMPL expression graphs.

[80] D. M. Gay, "More AD of nonlinear AMPL models: Computing hessian information and exploiting partial separability," in *Computational Differentiation: Techniques, Applications, and Tools*. SIAM, 1996, sparse Hessians via partial separability, no graph coloring.

[81] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[82] JuMP developers, "JuMP documentation," https://jump.dev/JuMP.jl/stable/, 2026.

[83] JuMP developers, "MathOptInterface documentation," https://jump.dev/MathOptInterface.jl/stable/, 2026.

[84] I. S. Duff and J. K. Reid, "The multifrontal solution of indefinite sparse symmetric linear equations," *ACM Transactions on Mathematical Software*, vol. 9, no. 3, pp. 302–325, 1983.

[85] J. W. Demmel, S. C. Eisenstat, J. R. Gilbert, X. S. Li, and J. W. H. Liu, "A supernodal approach to sparse partial pivoting," *SIAM Journal on Matrix Analysis and Applications*, vol. 20, no. 3, pp. 720–755, 1999.

[86] J. W. H. Liu, "The multifrontal method for sparse matrix solution: Theory and practice," *SIAM Review*, vol. 34, no. 1, pp. 82–109, 1992.

[87] E.-J. Im, K. Yelick, and R. Vuduc, "Sparsity: Optimization framework for sparse matrix kernels," *International Journal of High Performance Computing Applications*, vol. 18, no. 1, pp. 135–158, 2004.

[88] R. Vuduc, J. W. Demmel, and K. A. Yelick, "OSKI: A library of automatically tuned sparse matrix kernels," *Journal of Physics: Conference Series*, vol. 16, pp. 521–530, 2005.